

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
**«ПЕРМСКИЙ ГОСУДАРСТВЕННЫЙ ГУМАНИТАРНО-ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ»**

ФАКУЛЬТЕТ ИНФОРМАТИКИ И ЭКОНОМИКИ

Кафедра прикладной информатики, информационных систем и технологий

Курсовая работа
по модулю «Учебно-исследовательской и проектной деятельности»

**ИСПОЛЬЗОВАНИЕ АЛГОРИТМА WAVE FUNCTION
COLLAPSE ДЛЯ АВТОМАТИЧЕСКОЙ ГЕНЕРАЦИИ
УРОВНЕЙ ИГРЫ ЖАНРА ГОЛОВЛОМКА**

Работу выполнили:
обучающийся 1233 группы
направления подготовки
09.03.03 Прикладная информатика,
профиль «Прикладная информатика»
Штягина Дарья Николаевна

(подпись)

обучающийся 1237 группы
Направления подготовки
09.03.02 «Информационные системы
и технологии», профиль
«Информационные системы»
Плешкан Антон Андреевич

(подпись)

«Допущена к защите»
И.о. Заведующий кафедрой ПриИСиТ
доцент Кузнецов А.Г.

(подпись)

« ____ » _____ 2025 г.

Оценка _____

Руководитель _____

(подпись)

Руководитель:
преподаватель
Худорожков Лев Юрьевич

(подпись)

ПЕРМЬ
2025

Содержание

ВВЕДЕНИЕ.....	4
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1. Анализ существующих алгоритмов генерации текстур	7
1.2. Выбор инструментов разработки	11
Вывод по первой главе	13
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И ГЕЙМДИЗАЙН	15
2.1. Проектирование игрового процесса.....	15
2.1.1. Концепция.....	15
2.1.2. Сюжет.....	15
2.1.3. Механики игры.....	16
2.1.4. Игровой мир	18
2.1.5. Дизайн игры.....	20
2.1.6. Персонажи	21
2.2. Проектирование пользовательского интерфейса	23
2.2.1. Уровни интерфейса.....	23
2.2.2. Макет интерфейса	24
2.2.3. Взаимодействие с интерфейсом	25
Вывод по второй главе	26
ГЛАВА 3. РАЗРАБОТКА ИГРЫ.....	27
3.1. Разработка игрового процесса	27
3.1.1. Реализация управления персонажем.....	27
3.1.2. Взаимодействие с объектами.....	29
3.1.2. Разработка системы навыков персонажа.....	32

3.1.3. Создание 3D-моделей	35
3.2. Разработка алгоритма генерации уровней	38
3.2.1. Настройка ячеек и ограничений	39
3.2.2. Адаптация WFC для 3D-лабиринтов	40
3.3. Разработка пользовательского интерфейса.....	47
3.4. Аprobация.....	49
Вывод по третьей главе	51
ЗАКЛЮЧЕНИЕ	53
СПИСОК ЛИТЕРАТУРЫ	54
ПРИЛОЖЕНИЕ 1	55
ПРИЛОЖЕНИЕ 2	58
ПРИЛОЖЕНИЕ 3	63

ВВЕДЕНИЕ

В последние годы игровая индустрия сталкивается с необходимостью постоянного повышения комплексности и разнообразия контента, что особенно заметно на примере уровней в играх. Современные игры стремятся предоставить игрокам богатый и разнообразный опыт, однако ручная разработка каждого уровня требует от разработчиков огромных затрат времени и ресурсов. Эта проблема особенно актуальна для игр жанра головоломка, где ключевую роль играет тонкий баланс сложности и увлекательности, требующий скрупулезной ручной настройки.

Для решения этой задачи активно развивается направление процедурной генерации контента. Одним из перспективных алгоритмов в этой области является Wave Function Collapse (WFC). В отличие от многих других алгоритмов синтеза текстур, WFC способен создавать органичные и логичные уровни, практически неотличимые от созданных вручную. Гибкость алгоритма в настройке правил и ограничений открывает перед геймдизайнерами широкие возможности для создания уникальных и креативных механик головоломок.

Данный алгоритм был разработан М. Гуминым [1] для создания «плиточных» изображений на основе простой конфигурации или образцов изображений. По сравнению с более ранними алгоритмами синтеза текстур [2], WFC гарантирует, что выходные данные содержат только те шаблоны, которые присутствуют во входных данных.

Практическая ценность исследования WFC в контексте генерации уровней головоломок заключается в возможности оптимизации процесса разработки игр. Создание прототипа генератора уровней на основе WFC позволит не только ускорить и упростить процесс создания уровней, но и откроет новые горизонты для гейм-дизайна, позволяя реализовать идеи, трудноосуществимые при ручной разработке. Таким образом, исследование

WFC является не только актуальной задачей, но и вкладом в развитие алгоритмической генерации контента, демонстрируя потенциал и ограничения данного подхода на практике.

Целью данной работы является разработка игры с применением Wave Function Collapse для автоматического создания уникальных уровней.

Объектом исследования является процедурная генерация уровней для игр.

Предмет исследования – алгоритм Wave Function Collapse как инструмент генерации уровней со спецификой жанра головоломки.

В соответствии с объектом, предметом и целью данной работы были поставлены следующие задачи:

1. Проанализировать существующие алгоритмы, применяемые для генерации данных.
2. Проанализировать особенности алгоритма WFC и его применимость для генерации уровней.
3. Разработать и реализовать прототип генератора уровней на основе WFC для выбранного типа головоломки.
4. Провести тестирование и анализ сгенерированных уровней на предмет играбельности, разнообразия и соответствия заданным ограничениям.
5. Применить созданный прототип для создания уровня в игре-головоломке.

Методы исследования: системный анализ, (эксперимент), изучение и анализ литературы, описание, (моделирование).

Практическая значимость работы заключается в создании игры, способной автоматически формировать уровни с уникальными характеристиками, что улучшает пользовательский опыт и уменьшает затраты времени и ресурсов на ручное проектирование. Это особенно важно для игр жанра головоломка, где структура уровня напрямую влияет на сложность и увлекательность игрового процесса. Полученные результаты могут быть

применены не только в разработке игр, но и в других областях, где требуется автоматизация генерации контента, таких как архитектурное моделирование, визуализация данных или создание виртуальных сред. Работа демонстрирует потенциал использования алгоритма WFC для повышения креативности и эффективности в разработке интерактивных продуктов, способствуя расширению подходов к генерации контента в цифровых средах.

В настоящее время тенденция применения алгоритмов генерации для создания контента в игре популярна, поэтому актуальность разрабатываемого программного продукта довольно высока, как она отражает современные методы в разработке игр и предлагает новый подход к созданию уникальных уровней. Например, экспериментальный градостроительный симулятор Townscaper вышел летом 2020 года и за полгода собрал в Steam более 7000 положительных отзывов. Это песочница, в которой игрок строит город на тайловой сетке с уникальным узором, не имея под рукой конструктора. Нужно просто указать место для блока, а алгоритм автоматически превратит его в дом, арку, лестницу, мост или двор, бесшовно встраивая в окружение [3].

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Данная глава посвящена алгоритму Wave Function Collapse (WFC) – методу процедурной генерации контента, использующему принципы квантовой механики для создания разнообразных и организованных структур. Будут проанализированы альтернативные методы генерации, сравнены с WFC и обоснован выбор данного алгоритма для решения поставленных задач. Также будет выбран основной инструмент разработки.

1.1. Анализ существующих алгоритмов генерации текстур

В контексте процедурной генерации карт, где тайлы представляются узлами графа, классические алгоритмы поиска кратчайшего пути, такие как A* [4] и Дейкстры, демонстрируют ограниченную эффективность (см. рис. 1).

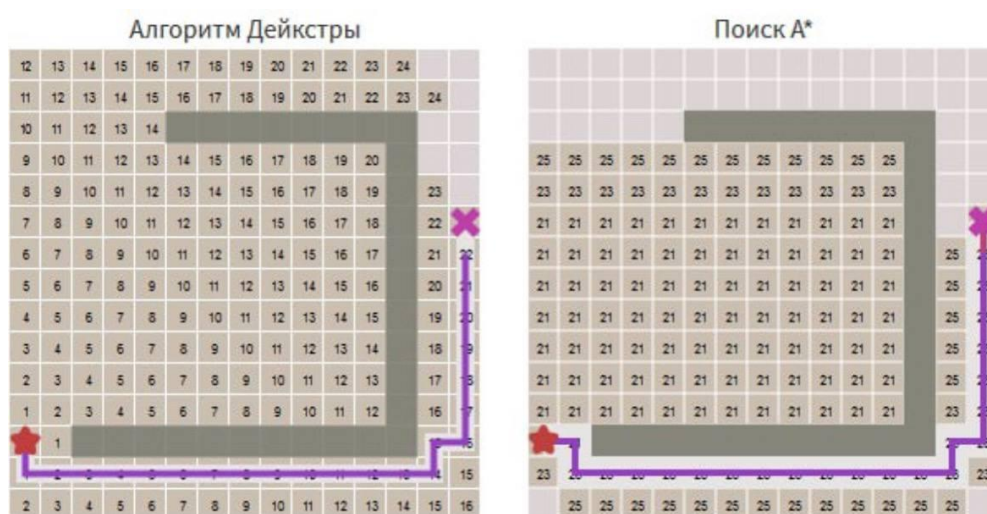


Рис. 1. Алгоритм Дейкстры и A*

Несмотря на применимость для поиска оптимальных траекторий, генерируемые ими структуры зачастую характеризуются предсказуемостью и

повторяемостью паттернов, что снижает их пригодность для создания реалистичных игровых уровней.

Метод рекурсивного деления [5], основанный на принципе "разделяй и властвуй", проявляет себя более успешно в задачах генерации органических форм. Последовательное деление пространства на меньшие области позволяет создавать сложные структуры, однако, контроль над финальным результатом может быть затруднен (см. рис. 2).

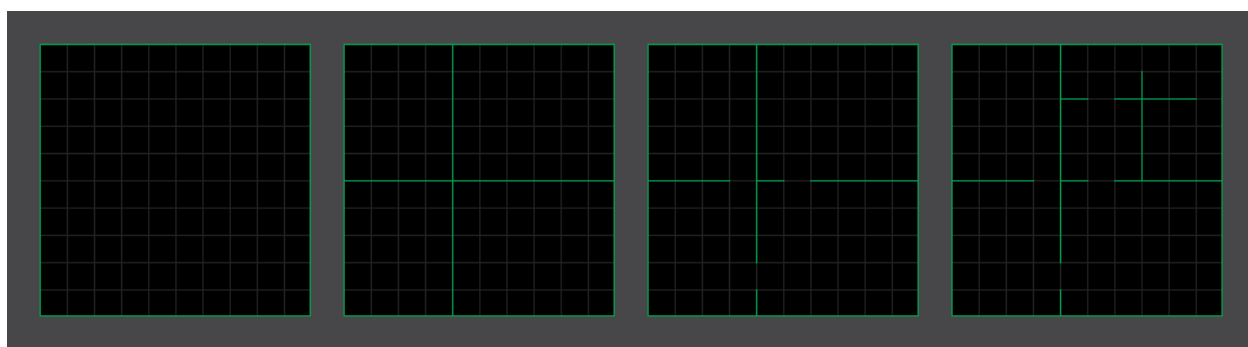


Рис. 2. Метод рекурсивного деления.

Кроме того, данный метод подвержен возникновению нежелательных артефактов на границах деления, что негативно сказывается на визуальном качестве генерируемых карт.

Алгоритм Прима [6], широко применяемый для генерации лабиринтов, также находит применение в процедурной генерации карт. Создаваемые с его помощью структуры отличаются связностью и ветвлением, однако разнообразие генерируемых паттернов остается ограниченным (см. рис. 3).

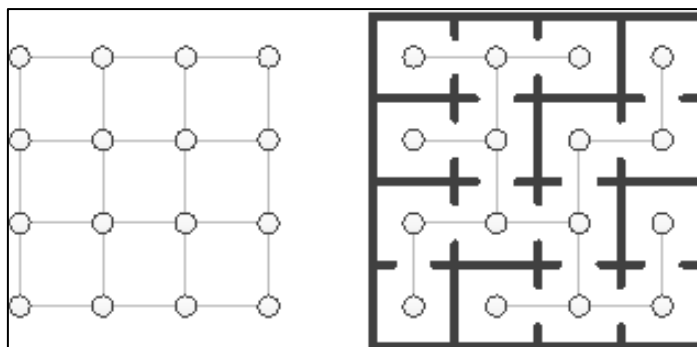


Рис. 3. Алгоритм Прима

В сравнении с вышеперечисленными методами, алгоритм Wave Function Collapse (WFC) демонстрирует большую гибкость и эффективность в создании организованных и разнообразных карт. Алгоритм основан на идее представления каждого тайла в виде набора "возможных состояний", которые постепенно "коллапсируют" в процессе итеративной обработки с учетом заданных ограничений и влияния соседних тайлов (см. рис. 4). Такой подход, обозначаемый как "распространение ограничений", позволяет генерировать карты, характеризующиеся высокой степенью когерентности и сложности, что делает WFC перспективным инструментом для разработки игр и других приложений, требующих создания процедурного контента.

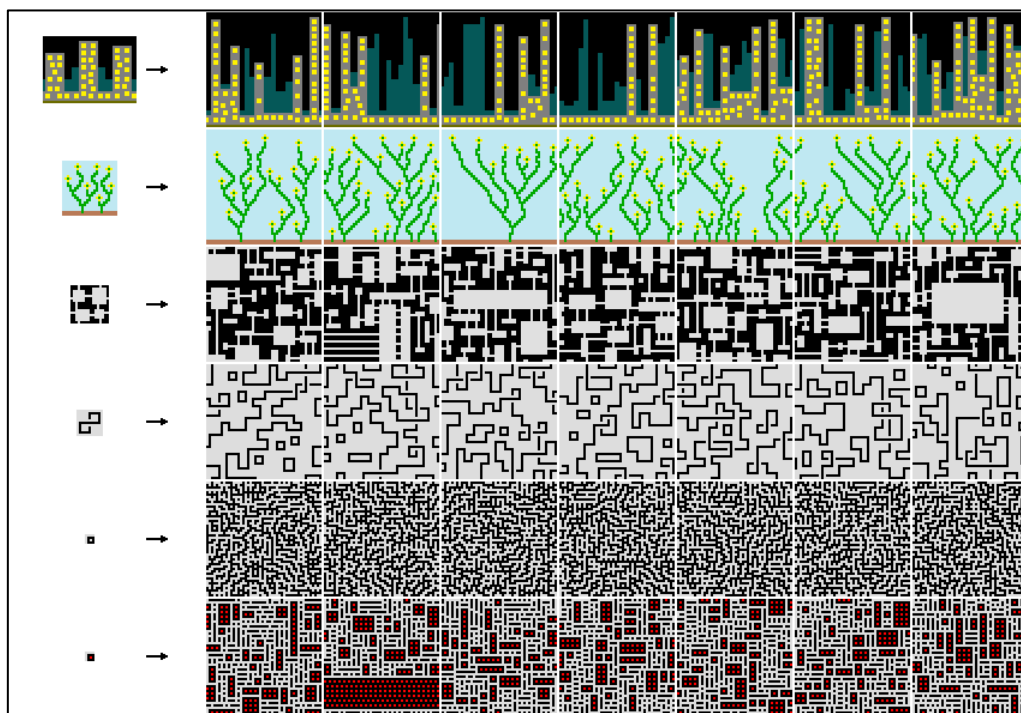


Рис. 4. WFC

WFC не просто генерирует случайные паттерны, он улавливает суть входных данных – будь то образец карты, набор правил или тайлсет - и создаёт бесконечные вариации, сохраняющие стилистическую целостность. Способность алгоритма WFC к обучению на основе входных данных и генерации контента, обладающего высокой степенью сходства с объектами, созданными человеком, определяет его значительный потенциал в качестве

инструмента для разработчиков компьютерных игр, художников, а также специалистов, работающих в области синтеза визуального контента.

Таким образом, наибольший потенциал для генерации комплексных и визуально убедительных карт принадлежит алгоритму Wave Function Collapse (WFC) благодаря его способности к итеративной обработке и учету глобальных ограничений в процессе генерации.

Таблица 1 – Анализ алгоритмов генерации карт

Алгоритм	Плюсы	Минусы
Поиск пути (A*, Dijkstra)	1. Создание органических узоров, напоминающих прожилки. 2. Относительная простота реализации.	1. Ограниченность в создании сложных паттернов. 2. Высокая вычислительная сложность, особенно для больших разрешений.
Рекурсивное деление	1. Простота реализации. 2. Легкий контроль над плотностью и сложностью генерируемых структур.	1. Однообразность и искусственность генерируемых карт. 2. Неэффективность для органических или нерегулярных паттернов.
Алгоритм Прима	1. Создание естественных, связанных структур. 2. Относительная простота реализации.	1. Ограниченность в создании сложных, детализированных карт. 2. Склонность к созданию повторяющихся паттернов.
WFC	1. Универсальность: генерация широкого спектра карт (от органических до геометрических, от повторяющихся до уникальных). 2. Высокая организованность: учет глобальных зависимостей для создания когерентных и реалистичных паттернов. 3. Гибкость: возможность точной настройки параметров генерации и задание желаемых свойств карт.	Может быть вычислительно затратным, особенно для больших карт с комплексом правил.

1.2. Выбор инструментов разработки

WFC позволяет генерировать сложные, процедурно создаваемые уровни, обеспечивая высокую реиграбельность и непредсказуемость игрового процесса. На первый взгляд мощные инструменты вроде Unreal Engine кажутся привлекательными, но интеграция WFC в них может оказаться сложнее, чем предполагалось.

Unreal Engine, известный своей реалистичной графикой и широкими возможностями, представляет собой сложную экосистему. Хотя визуальный скриптинг на Blueprint упрощает некоторые задачи, интеграция алгоритма, подобного WFC, требует глубокого понимания C++ и архитектуры движка. Это может стать проблемой, особенно для небольших команд или разработчиков-одиночек. Настройка WFC в Unreal Engine потребует значительных временных затрат и глубокого изучения, что может отвлечь от непосредственно разработки геймплея и дизайна уровней.

На данном движке была разработана следующая игра - Eternal Mist (см. рис. 5). В этой тактической стратегии разработчики применили WFC для генерации более 100 уникальных уровней, что позволило значительно сократить время на их создание и разнообразить игровой процесс.

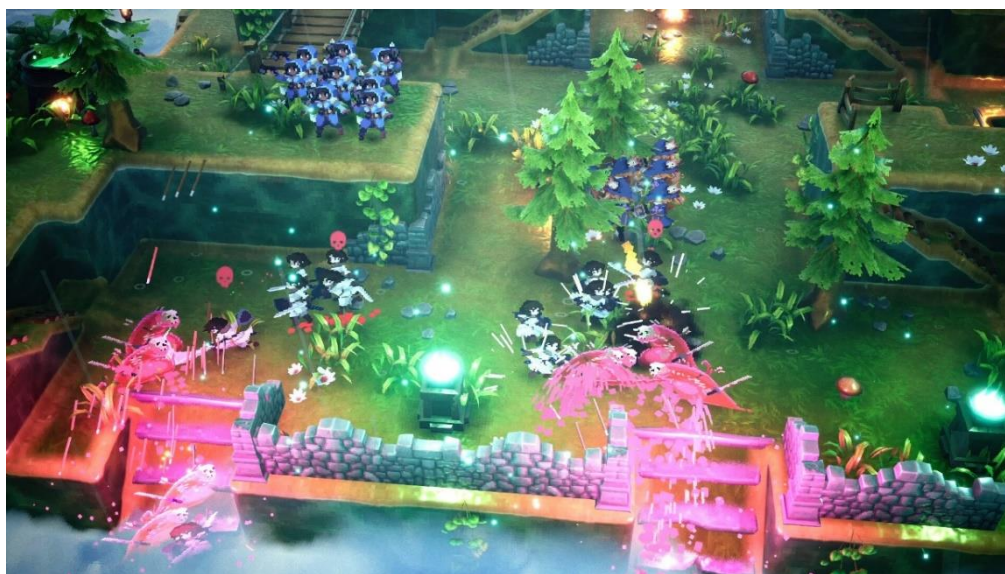


Рис. 5. Скриншот из игры Eternal Mist

Godot, бесплатный и открытый движок, предлагает более простой и интуитивно понятный подход к разработке. Его легкость в освоении и дружелюбный интерфейс делают его привлекательным вариантом для начинающих разработчиков и небольших проектов. Однако, несмотря на постоянное развитие, 3D-функционал Godot пока отстает от таких гигантов, как Unreal Engine и Unity. Это может стать ограничивающим фактором при создании сложных трехмерных головоломок с высокой детализацией и визуальной насыщенностью, особенно при использовании ресурсоемкого алгоритма WFC. Оптимизация производительности в Godot для подобных проектов может потребовать дополнительных усилий.

Unity сочетает в себе мощный функционал с относительной простотой использования. Данный движок предлагает богатый набор инструментов, включая удобный редактор уровней, развитую систему физики, поддержку разнообразных платформ и мощный API, облегчающий интеграцию сложных алгоритмов, таких как WFC. Работа с C# в Unity более доступна для широкого круга разработчиков по сравнению с C++ в Unreal Engine.

Это удобный бесплатный инструмент для начинающих разработчиков, в нем можно создавать проекты в одиночку. На этом движке созданы проекты Genshin Impact, Hearthstone, Outlast, Cuphead, Pokemon GO и другие популярные игры.

Но главное преимущество Unity в работе с WFC — это его гибкость и открытость, подкрепленная огромным сообществом и богатой экосистемой. Unity Asset Store предоставляет доступ к множеству готовых решений, ассетов и примеров реализации WFC. Это позволяет разработчикам значительно сократить время разработки, быстро проектировать идеи и экспериментировать с различными подходами. Активное сообщество Unity всегда готово помочь с возникающими вопросами и поделиться опытом, что делает процесс разработки более плавным и эффективным.

Игра Townscaper применяет упрощенную версию WFC для автоматической генерации городских построек и улиц. Благодаря этому

алгоритму создаются визуально гармоничные структуры с минимальным вмешательством игрока (см. рис. 6).



Рис. 6. Скриншот из игры Townscaper

Таким образом, для создания трехмерных головоломок с использованием WFC, Unity предлагает оптимальный баланс мощности, гибкости, доступности и поддержки сообщества. Он позволяет разработчикам сосредоточиться на самом важном – на создании увлекательного геймплея и дизайна уровней, не тратя чрезмерное время на сложности интеграции WFC. Именно это делает Unity идеальным выбором для данного типа проектов.

Вывод по первой главе

В рамках первой главы был проведен сравнительный анализ алгоритмов процедурной генерации контента, включая алгоритмы A*, Дейкстры, метод рекурсивного деления, а также алгоритм Прима. Установлено, что данные алгоритмы, несмотря на свою эффективность в отдельных случаях, демонстрируют ряд ограничений применительно к задачам генерации вариативных и структурно организованных игровых уровней. На основании

проведенного анализа, в качестве наиболее перспективного метода был выбран алгоритм Wave Function Collapse (WFC), демонстрирующий высокую степень гибкости, способность к обучению на основе входных данных и учету глобальных ограничений в процессе генерации, что позволяет создавать сложные и визуально убедительные структуры. В качестве оптимального инструмента для реализации проекта разработки трехмерной головоломки с использованием алгоритма WFC выбран движок Unity, обладающий необходимым уровнем гибкости, производительности и широким набором инструментов, включая поддержку языка программирования C#, функциональный редактор уровней и доступ к развитому сообществу разработчиков.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И ГЕЙМДИЗАЙН

2.1. Проектирование игрового процесса

2.1.1. Концепция

Игра представляет собой исследование 3D-лабиринтов, где основной упор сделан на решение пространственных головоломок. Отличительной чертой являются двухсторонние лабиринты, существующие в двух параллельных плоскостях, которые связаны между собой переходами. Игрок исследует лабиринты, полные препятствий, таких как ловушки, исчезающие платформы.

Цель игры – решить головоломки на уровне и пройти до финиша. Данная задача усложнена: на пути встречаются различные враги, от стандартных монстров до уникальных боссов, обладающих уникальными способностями и требующих тактического подхода.

2.1.2. Сюжет

Начало: Главный герой, Альбедо, решается пройти испытания в лабиринте, созданном древним божеством Орионом, который ищет достойного преемника. Этот мир, некогда процветающий, теперь балансирует на грани хаоса, разделённый на четыре континента, каждый из которых страдает от влияния одного из Всадников Апокалипсиса: Голода, Войны, Чумы и Смерти. Альбедо, бывший воин, предавший свой народ ради семьи, но потерявший их из-за собственной нерешительности, теперь стремится искупить вину и обрести силу, чтобы изменить прошлое.

Середина: По мере продвижения Альбедо сталкивается с каждым Всадником. В Азракане, опустошённом Голодом, время ограничено, а съеденные ядовитые грибы искажают зрение. В Вальтарии, охваченной Войной, зеркальные коридоры создают иллюзии предательства, а число врагов растёт с каждой минутой. Луменис, поражённый Чумой, заставляет Альбедо бороться с роями насекомых и ядами, искажающими реальность, а в Нордхейме, царстве Смерти, его преследуют духи павших воинов, а холод замедляет каждый шаг.

Конец: Достигнув сердца лабиринта, Альбедо узнаёт, что Всадники — всего лишь заложники системы Ориона, который, разочаровавшись в собственной непогрешимости, ищет того, кто сможет объединить разум и сердце. В финальной битве Альбедо решает уничтожить Всадников, используя их же силы против них. В итоге Орион растворяется в энергии лабиринтов, передавая Альбедо свою силу.

2.1.3. Механики игры

Механики персонажа:

Правая кнопка мыши: передвижение.

Левая кнопка мыши: атака нимбом (см. рис. 7) в небольшом радиусе. Нужна для атаки по маленьким приспешникам, которая моментально убивает их (попадает по всем противникам вблизи).

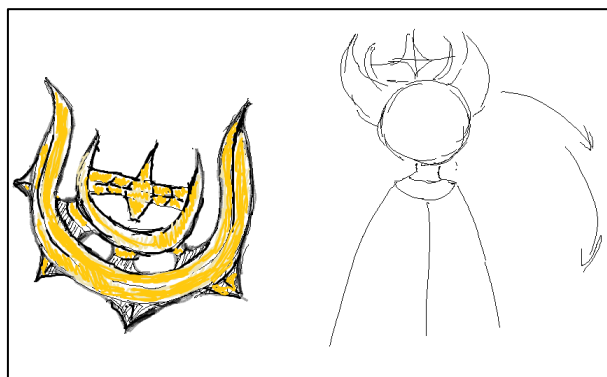


Рис. 7. Концепт-арт нимба

Навыки персонажа:

1. *Пробел*: основная кнопка взаимодействия с интерактивными предметами в игре.
2. *Alt*: режим просмотра карты. Увеличивает радиус обзора игрока на 5 секунд, перезарядка 15 секунд, количество использований на уровень - 3 раза.
3. *Ctrl*: Активация невидимости на 2 секунды, перезарядка 30 секунд.
4. *Shift*: Телепортация на небольшое расстояние, ограниченное количество использований на уровень - максимум 5 раз.
5. *Z,X,C*: Взаимодействие с метками, всего 5 штук

Условие активации навыков: активируются по прохождению уровней.

Особенности: игрок может перемещаться в радиусе 12,5 единиц вокруг персонажа, который будет выделен (подсвечен белым цветом, весь фон кроме блоков будет затемнен на 50%), сделано для того, чтобы было удобнее проходить уровни. У игрока будет одно сердце, состоящее из двух половин, один удар - минус половина, второй – проигрыш. Восстановить можно выпив зелье на уровне.

Механика приспешников: Являются обычными противниками. Могут перемещаться по уровню (бегать за игроком).

Есть несколько видов противников:

1. Взрывающийся - подбегает к игроку и моментально взрывается, снимая две единицы здоровья.
2. Обычный - приспешник, что может подойти к игроку и ударить его на 1 единицу здоровья.

Механика всадников:

1. Смерть – если игрок находится в прямой видимости, то он моментально умирает.

2. Чума – на локации будет пролетать рой чумы (черный дым), нужно не попасться, иначе на игрока будут наложены ослабления. Если попасться под удар роя в первый раз, то скорость передвижения и видимость уменьшится. Если попасться во второй раз, то игроку наносится урон равный одной единице здоровья.
3. Война – к изначальным врагам на уровне этот всадник может добавлять еще больше врагов (максимум 10). Ставит он их в случайные локации, ближе или дальше к игроку.
4. Голод – ставится лимит времени на уровень - 3 минуты. Появляются особые предметы – грибы. Подобрал гриб, он восстановит время, которое рассчитывается в зависимости от приблизительно оставшегося времени при среднем темпе прохождения.

Механики лабиринта (ловушки):

1. Замедления - сразу при наступлении на ловушку, игрок замедляется на 50% на 2 секунды, откат 1 секунду, потом активируется снова.
2. Оцепенения - сразу при активации ловушки, игрок не может двигаться 2 секунды, откат 5 секунд, потом активируется снова.
3. Меч - отнимает одну единицу здоровья при воздействии на игрока.

На уровнях будут встречаться порталы, которые перемещают игрока на противоположную сторону платформы. Эта сторона является продолжением лабиринта и является частью уровня.

2.1.4. Игровой мир

Игровой мир представляет собой абстрактное пространство, где каждый уровень формируется из множества отдельных тайлов. Вместо фиксированной карты или заранее определённых локаций, мир существует как череда уникальных головоломок. Такая структура обеспечивает непрерывное ощущение новизны и непредсказуемости.

Также он представляет собой минималистичную вселенную, выполненную в мрачной и атмосферной стилистике. Пространство уровней состоит из изометрических тайлов, объединяющих элементы архитектуры с элементами загадок. Каждый уровень (см. рис. 8) — это замкнутая локация, погружённая в полумрак, с акцентом на контрастное освещение, создаваемое факелами и светильниками.

Центральное место часто занимают загадочные структуры — такие как порталы или алтари, которые связаны с ключевыми механиками игры. Атмосфера уровней подчёркивается визуальными деталями: изношенными поверхностями, мерцанием света и тенями.

Музыкальное сопровождение поддерживает атмосферу, усиливая напряжение или погружая игрока в состояние размышлений.

Основные тайлы:

1. Начальный (алтарь начала)
2. Финальный (алтарь завершения)

Дополнительные тайлы:

1. Прямой
2. Конечный
3. С поворотом
4. Перекрёсток
5. Три прохода

Особенности:

1. Дверь
2. Ключ
3. Портал
4. Ваза (замедление)
5. Меч
6. Камень (оцепенение)

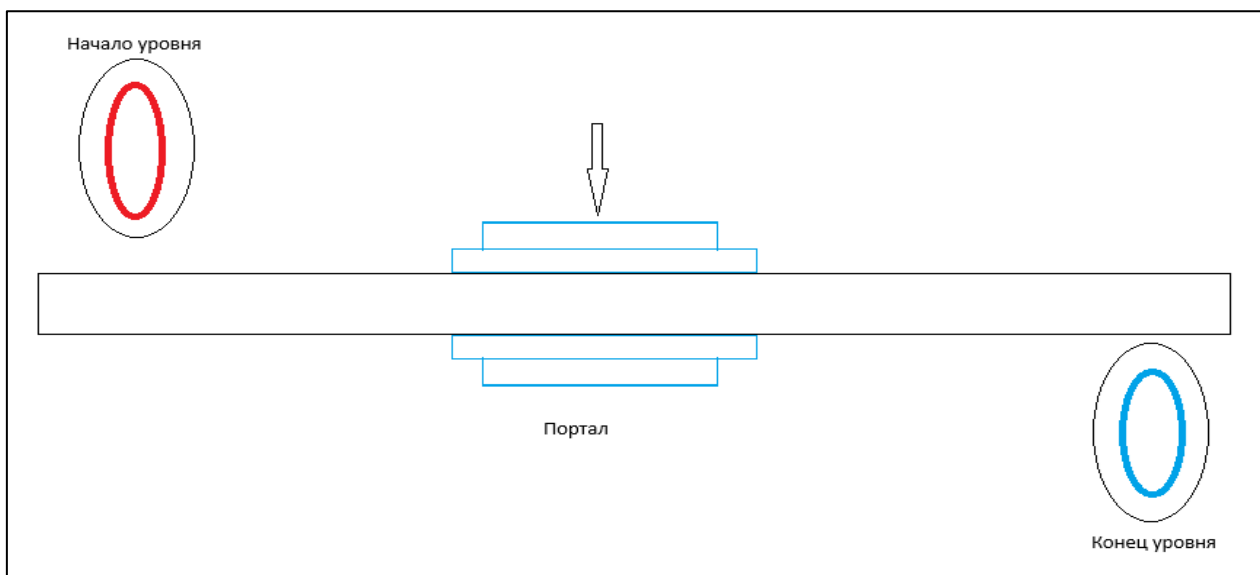


Рис. 8. Схема уровня

2.1.5. Дизайн игры

Визуальный стиль игры можно охарактеризовать как минималистичный, мрачный и загадочный, с акцентом на атмосферу таинственности.

Уровни выполнены в изометрической проекции с упором на геометрические формы. Преобладают темные оттенки, которые создают ощущение пустоты и изоляции. Износ поверхностей — трещины, царапины и грубые текстуры — подчеркивает древность и запустение пространства. Мягкое освещение от свечей и магических объектов служит важными визуальными акцентами, оживляя уровни и направляя внимание игрока.

Арт выполнен в минималистичной 3D-стилистике (low poly) с низким количеством деталей, что добавляет современный, абстрактный тон. Основное внимание уделено освещению, теням и текстурным контрастам. Локации создают баланс между простотой и детализацией, делая их одновременно стильными и загадочными.

Звуковые эффекты:

1. Сгорание двери
2. Взмах нимбом и его попадание по врагу
3. Звуки кликов
4. Поднятие ключа
5. Фоновая музыка в меню
6. Фоновая музыка на уровнях
7. Телепортация
8. Невидимость

2.1.6. Персонажи

Главный герой (см. рис. 9) – персонаж, которым управляет игрок. Пришел пройти испытания Бога, чтобы заполучить его силу. Отличительная особенность – длинный синий плащ и светлая кожа.

Приспешники (см. рис. 9) – основные враги, которых встречает Альбедо во время своего путешествия в лабиринтах. Являются порождением Бога и призваны, чтобы испытывать участников испытания.



Рис. 9. Концепт-арт главного героя и приспешников

Всадники смерти (см. рис. 10) представляют собой ключевых персонажей игры, выполняющих роль слуг высшего божественного существа.

1. Смерть - Образ классического воплощения смерти с капюшоном и косой. Тёмный плащ и череповидное лицо создают ассоциации с древними мифологическими представлениями о смерти.
2. Чума - Персонаж в маске, напоминающей чумных докторов, что ассоциируется с болезнями и очищением. Одет в зелёную мантию и стилизованную маску.
3. Война - Красный наряд и копье. Этот персонаж символизирует конфликт и испытания силы.
4. Голод - Худощавое тело подчеркивает его природу и способности. Связан с управлением временем.



Рис. 10. Концепт всадников смерти

Бог (см. рис. 11) представляет собой высшее существо, которое управляет миром и направляет действия своих слуг — Всадников смерти. Его образ олицетворяет абсолютную власть, неоспоримую мудрость и одновременно скрытую таинственность. Бог является не только центром сюжета, но и основным источником испытаний для игрока.

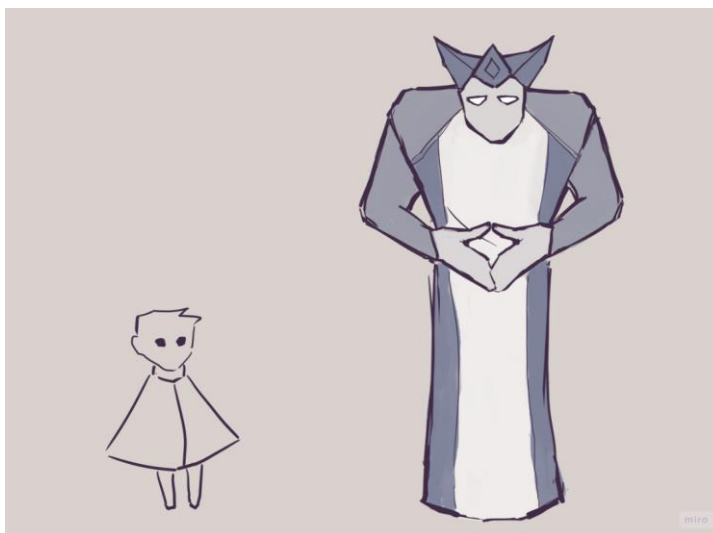


Рис.11. Бог рядом с главным героем

2.2. Проектирование пользовательского интерфейса

Проектирование пользовательского интерфейса (UI) напрямую влияет на удобство и взаимодействия игрока с игровым миром. В данной работе интерфейс будет разработан с учетом минималистичного стиля игры, чтобы обеспечить погружение игрока в атмосферу.

2.2.1. Уровни интерфейса

Пользовательский интерфейс игры будет разделен на несколько уровней, каждый из которых выполняет определенные функции:

Стартовый экран: на этом экране игроку предоставляется возможность выбора уровня. Поле выбора уровня включает номера уровней, что позволяет игроку легко начать игру с выбранным этапом. Кнопка "Нажмите пробел чтобы начать" инициирует начало игры, а кнопка "Выйти" позволяет покинуть игру.

Меню в игре: во время игры игрок может открыть меню, где доступны опции "Продолжить", "Начать заново", "Выйти", "В главное меню", а также регулирование громкости музыки и описание кнопок способностей. Это обеспечивает гибкость управления игровым процессом и позволяет игроку при необходимости перезапустить уровень или вернуться в главное меню.

Экран смерти: в случае поражения игроку предлагается выбор: "Выйти", "Начать заново" или "В главное меню". Этот экран помогает игроку быстро восстановиться после неудачи и продолжить игру.

2.2.2. Макет интерфейса

Макет пользовательского интерфейса, представленный на Рисунках с 12 по 14, демонстрирует расположение основных элементов UI. Интерфейс будет выполнен в темных оттенках, что соответствует общей атмосфере игры.

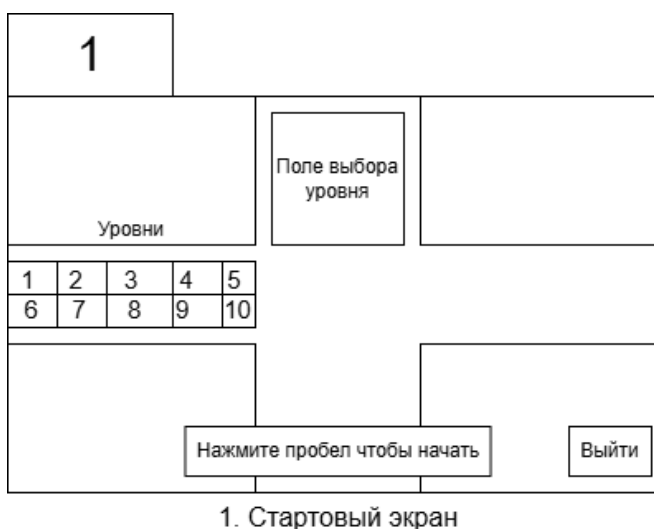
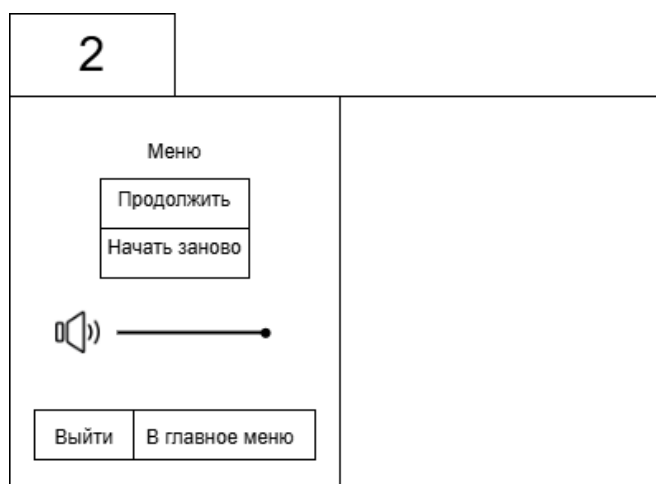
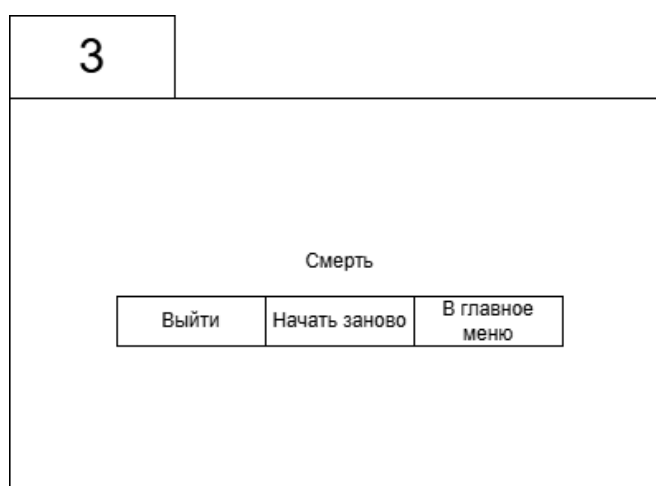


Рис. 12. Стартовый экран



2. Меню в игре

Рис. 13. Меню в игре



3. Экран смерти

Рис. 14. Экран смерти

2.2.3. Взаимодействие с интерфейсом

Интерфейс игры разработан с учетом удобства взаимодействия. Все кнопки и элементы управления реагируют на действия игрока, обеспечивая плавный и понятный процесс навигации. Например, выбор уровня осуществляется простым нажатием на соответствующий номер, а переход между сценами происходит с плавной анимацией затемнения экрана.

Вывод по второй главе

В рамках второй главы был проведен анализ и проектирование ключевых аспектов игрового процесса и пользовательского интерфейса для игры, основанной на алгоритме Wave Function Collapse (WFC). Была разработана концепция игры, включающая уникальные механики, такие как двухсторонние лабиринты, взаимодействие с различными типами врагов и использование пространственных головоломок. Особое внимание уделено проектированию игрового мира, который представляет собой динамически генерируемые уровни, созданные с помощью WFC, что обеспечивает высокую реиграбельность и разнообразие игрового опыта.

Проектирование пользовательского интерфейса было выполнено с учетом минималистичного и мрачного стиля игры, что способствует погружению игрока в атмосферу таинственности и вызова. Интерфейс разделен на несколько уровней, включая стартовый экран, меню в игре и экран смерти, что обеспечивает удобство навигации и интуитивность взаимодействия.

ГЛАВА 3. РАЗРАБОТКА ИГРЫ

3.1. Разработка игрового процесса

В рамках данного раздела описаны этапы реализации ключевых механик игры, включая управление персонажем, взаимодействие с объектами, систему навыков, противников, ловушек и порталов.

3.1.1. Реализация управления персонажем

Персонаж (Альбедро) контролируется игроком через стандартные средства ввода: клавиатуру и мышь. Основные действия:

Движение: Перемещение осуществляется с помощью правой кнопки мыши. Для реализации использовался Unity InputSystem, с помощью которого определяется позиция курсора на экране и состояние кнопок мыши.

Камера: Она следит за персонажем в изометрической проекции. Добавлено сглаживание движения камеры для более плавного передвижения обзора.

Физика: Использование компонентов Rigidbody, Collider и NavMesh для взаимодействия с ландшафтом и объектами уровня

Для обеспечения реалистичного взаимодействия персонажа, врагов и объектов с ландшафтом и между собой в игре были использованы ключевые компоненты физического движка Unity: Rigidbody, Collider и NavMesh. Эти элементы позволили реализовать корректные столкновения, гравитацию, движение по сложной поверхности и навигацию врагов.

Rigidbody был использован для корректного взаимодействия между интерактивными объектами.

Компоненты Collider задают форму объекта для физических взаимодействий. Для статических объектов (платформы) применялись

BoxCollider, а для персонажа — CapsuleCollider. Это обеспечило точное определение столкновений.

Триггеры: Ловушки (замедление, оцепенение) работали как триггеры. При пересечении игроком триггерной зоны активировался соответствующий эффект через события OnTriggerEnter.

Для реализации движения по уровню использовался NavMesh — система навигации, которая позволяет NPC находить оптимальные пути к цели. Этапы настройки:

Создание NavMesh (листинг 1): Статические объекты (платформы) маркировались как Static и участвовали в построении NavMesh через меню Window – AI – Navigation. Это сформировало «карту доступности» для врагов. Для генерируемого уровня был создан скрипт, который в режиме реального времени протраивал NavMesh на тайлы.

```
private void Awake()
{
    if (surfaces.Length != 2) {
        Debug.Log("Incorrect Navmesh surface count in
        MeshGenerator.cs (NavMesh manager must have 2 surfaces)");
        return;
    }
    GameEvents.current.onPlatformDestroyed += RebuildMesh;
    if (WFCGenerator.instance != null)
        WFCGenerator.instance.onWFCFinished += RebuildMesh;
}
private void RebuildMesh() {
    GameObject.Find("Player").GetComponent<NavMeshAgent>().enabled =
false;
    for (int i = 0; i < surfaces.Length; i++) {
        surfaces[i].BuildNavMesh();
    }
    GameObject.Find("Player").GetComponent<NavMeshAgent>().enabled
= true;
}
```

Листинг 1. Генерация NavMesh

Настройка NavMeshAgent: К каждому врагу отдельно добавлялся компонент NavMeshAgent, который автоматически рассчитывал путь к игроку

с учетом препятствий. Скорость и ускорение агентов регулировались в инспекторе (см. рис. 15 и рис. 16).



Рис. 15. Обычный враг



Рис. 16. Взрывающийся враг

3.1.2. Взаимодействие с объектами

Порталы: При активации игрок перемещается на противоположную сторону платформы. Скрипт (листинг 2) выключает NavMeshAgent, берет агент игрока, переворачивает и включает обратно.

```

void AnimateFlip(WorkingSurface surf){
    agent.ResetPath();
    Vector3 originalScale = transform.localScale;
    transform.DOScale(new Vector3(originalScale.x * 0.85f,
originalScale.y * 1.25f, originalScale.z * 0.85f), 0.1f)
        .SetEase(Ease.OutQuint)
        .OnComplete(() => {
            transform.DOScale(originalScale,
0.15f).SetEase(Ease.InQuint);
            FlipAndSetPosition();});
    audioSource.PlayOneShot(teleportSound);}
void FlipAndSetPosition(){
    agent.enabled = false;
    transform.Rotate(0f, 0f, 180f);
    transform.Translate(isFlipped ? Vector3.down * 3 : Vector3.up *
3, Space.Self);
    agent.enabled = true;}

```

Листинг 2. Перемещение через портал

Двери и ключи (листинг 3): Двери блокируют путь до получения соответствующего ключа.

```

void OpenDoor() {
    foreach (Material material in materials) {
        if (material.HasProperty("_Dissolve")) {
            GetComponent().Play();
            GetComponent<NavMeshObstacle>().enabled = false;
            DOTween.To(() => material.GetFloat("_Dissolve"), x
=> material.SetFloat("_Dissolve", x), 1, dissolveTime)
                .SetEase(Ease.InOutQuart);}}}
public void Interact(GameObject interactor) {
    if
(interactor.GetComponent<Interactor>().GetItemCount(InventoryItem.Key)
< 1) {
        GameEvents.current.NewMindBubble("I need to find a
key...", 3.5f);
        return;
    } else {
interactor.GetComponent<Interactor>().RemoveItemFromInventory(Invent
oryItem.Key, 1); }
        if (isOpen) return;
        isOpen = true;
        OpenDoor();
    }
}

```

Листинг 3. Скрипт открытия двери

Метод OnTriggerStay (листинг 4) обрабатывает постоянное нахождение игрока в зоне ловушки. При выполнении условий (игрок в зоне, время задержки эффекта истекло, способность «невидимость» не активна) воспроизводится звуковой эффект, а компонент ObjectEffects игрока получает дебафф SlownessEffect с заданными параметрами замедления (slowness) и длительностью (slownessTime). Время последнего применения эффекта обновляется для предотвращения спама.

```
private void OnTriggerStay(Collider collision)
{
    if (collision.gameObject.CompareTag("Player") && Time.time
    > lastSlownessTime + slownessDelay)
    {
        if
        (collision.gameObject.GetComponent<Abilities>().IsInvisible) return;
        if (!audioSource.isPlaying) audioSource.Play();
        collision.gameObject.GetComponent<ObjectEffects>().AddEffect(new
        SlownessEffect(slowness), slownessTime);
        lastSlownessTime = Time.time;
    }
}
```

Листинг 4. Ловушка замедления игрока

Метод OnTriggerEnter активирует ловушку оцепенения при контакте с игроком. Проверяются тег объекта (Player) и время задержки эффекта (stunDelay). При выполнении условий отменяются текущие анимации (DOTween.Kill), объект ловушки растворяется (DissolveThis), игрок получает дебафф StunEffect через компонент ObjectEffects, а сама ловушка восстанавливается через задержку (DOVirtual.DelayedCall).

```

private void OnTriggerEnter(Collider collision) {
    if (collision.gameObject.CompareTag("Player") && Time.time
    > lastStunTime + stunDelay){
        DOTween.Kill(this);
        resetTween?.Kill();
        DissolveThis();
        collision.gameObject.GetComponent<ObjectEffects>().AddEffect(ne
w StunEffect(), stunTime);
        resetTween = DOVirtual.DelayedCall(DissolveTime,
ResetTrap);}
    }
}

```

Листинг 5. Ловушка оцепенения игрока

3.1.2. Разработка системы навыков персонажа

Система навыков персонажа представляет собой один из ключевых элементов игрового процесса, обеспечивающий игроку возможность взаимодействия с миром и решения пространственных головоломок. Была реализована модульная система, включающая пять уникальных способностей.

Основой системы стал подход, сочетающий визуальную обратную связь (шейдерные эффекты, анимации) и строгие игровые ограничения (таймеры перезарядки, лимиты использования).

Разработка велась с применением DOTween [7] — популярного инструмента для создания анимаций в Unity, который упрощает работу с плавными переходами и динамическими изменениями свойств объектов. Название происходит от сочетания «Do Tween», где «tween» (сокращение от «in-between») обозначает промежуточные кадры между начальным и конечным состояниями анимации.

Основные возможности:

1. Анимация свойств: Позволяет плавно изменять числовые значения (позицию, масштаб, прозрачность, цвет и др.) за заданное время.

2. Easing-функции: Поддерживает разнообразные кривые анимации (линейные, с ускорением, замедлением, упругие эффекты), что делает движения более естественными.
3. Цепочки анимаций: Объединение последовательных или параллельных анимаций в цепочки для сложных сценариев.
4. Работа с материалами и шейдерами: Например, изменение параметров шейдера для эффекта «растворения» (как в реализации невидимости персонажа).
5. Отложенные вызовы: Запуск действий после завершения анимации через `DOVirtual.DelayedCall`.

Это позволило создать отзывчивую систему (приложение 2), где каждое действие игрока (активация навыка, перезарядка) мгновенно отражается на игровом процессе.

Были реализованы следующие навыки:

1. Атака нимбом
2. Невидимость
3. Просмотр карты
4. Метки (пилоны)
5. Телепортация

Невидимость активирует эффект «растворения» модели персонажа через шейдеры, сопровождаемый звуковым оформлением. Состояние длится заданное время, после чего модель постепенно возвращается в видимое состояние. Система автоматически запускает перезарядку, блокируя повторное использование до завершения таймера.

Расширенный обзор временно увеличивает зону видимости игрока, синхронизируясь с игровыми событиями для обновления интерфейса. Уведомления о перезарядке и звуковые подсказки (например, предупреждение о завершении способности) обеспечивают обратную связь.

Телепортация позволяет перемещать персонажа в точку, указанную курсором, с учетом ограничений навигационной сетки и радиуса действия.

Количество доступных телепортаций ограничено, но автоматически восстанавливается после перезарядки. Скрипт отслеживает позицию на поверхности, предотвращая перемещение между слоями (например, с «верхнего» на «нижний» уровень).

Данный скрипт (приложение 1) реализует механику управления пилонами — специальными объектами, которые игрок может размещать в игровом мире для решения головоломок или навигации. Система позволяет создавать ограниченное количество пилонов (задаётся параметром `maxPlacablePylons`), каждый из которых может быть активирован, удалён или использован для визуализации направлений.

При размещении пилонa (`PlacePylon`) объект создаётся на позиции персонажа с учётом ориентации игрового пространства (переворот на 180 градусов для «нижнего» уровня). Механика сброса пилонов (`ResetPylons`) удаляет все активные объекты, сопровождаясь звуковыми эффектами для обратной связи.

Функция показа направлений (`ShowDirections`) использует частицы и анимации (через `DOTween`), чтобы визуально соединить позицию персонажа с размещёнными пилонами, создавая динамические траектории. Это помогает игроку анализировать окружение и планировать перемещения.

Данный скрипт (приложение 3) реализует механику атакующего нимба, который сопровождает персонажа. Нимб динамически следует за игроком, плавно корректируя свою позицию и угол наклона в зависимости от движения и текущего слоя игрового пространства (верхний или нижний уровень).

При активации атаки (клик мышью) нимб превращается в поражающий снаряд: он вращается, устремляется к указанной точке и наносит урон врагам через коллайдер. Анимация атаки включает эффект «растворения» с использованием шейдеров, а также изменение интенсивности свечения, создавая драматичный визуальный контраст между фазой подготовки и ударным воздействием.

Система ограничений (перезарядка) и синхронизация с событиями (например, смерть игрока) обеспечивают баланс между агрессивным геймплеем и тактическим планированием. Плавные переходы, реализованные через DOTween, и адаптация к окружению (учет поверхности) подчеркивают интеграцию механики в процедурно сгенерированный мир, делая её неотъемлемой частью игрового опыта.

3.1.3. Создание 3D-моделей

На основе концепт-артов (см. рис. 9–11) были определены ключевые визуальные характеристики моделей. Главный герой Альбедро (см. рис. 17) получил длинный синий плащ. Всадники смерти (см. рис. 18) разрабатывались с акцентом на их символику: Смерть — коса и капюшон, Чума — стилизованная маска чумного доктора, Война — копье и красные тона, Голод — карманные часы.

Для создания моделей и анимаций использовалась программа Maya 2025.



Рис.17. Модель главного персонажа



Рис. 18. Модели всадников смерти

Модульные блоки лабиринта (см. рис. 19): прямые коридоры, повороты, перекрестки, порталы. Каждый блок имел четко заданные точки соединения, что критично как для работы WFC, так и для создания полноценной карты уровня.

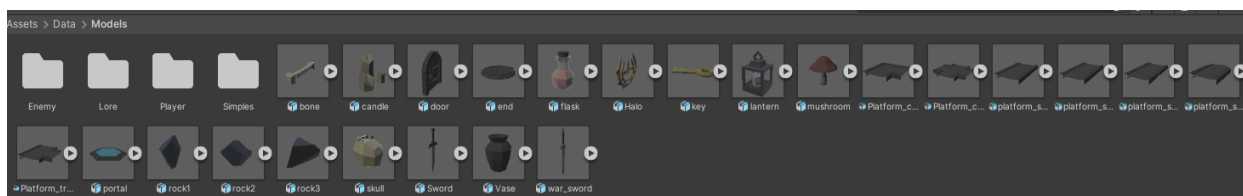


Рис.19. Атлас моделей игры

Модели создавались с материалами с акцентом на изношенность и мрачную палитру. Для сохранения стиля использовались приглушенные цвета (серые, темно-синие, красные). Например, материалы лабиринта включали трещины и царапины, усиливающие атмосферу древности.

Персонаж был снабжен упрощенным скелетом для базовой анимации (см. рис.20). Всадники смерти получили анимированные позы.

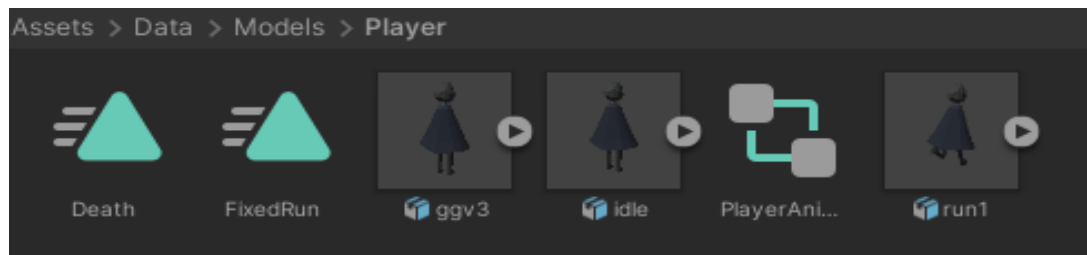


Рис. 20. Атлас анимаций главного героя

Приспешники были сделаны в виде дымки из геометрических фигур (см. рис. 21).

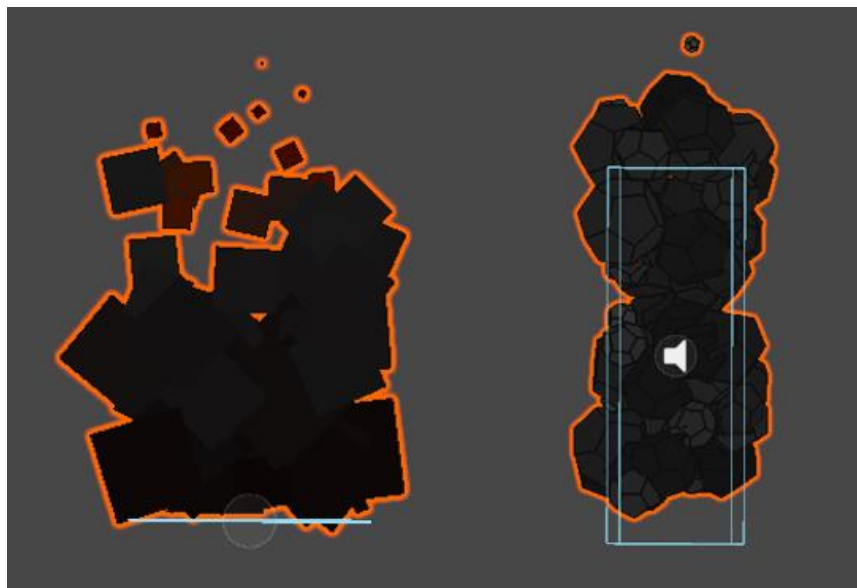


Рис.21. Внешний вид приспешников

Звуки (см. рис. 22) и иконки (см. рис. 23) распространялись в интернете с бесплатной лицензией автора.

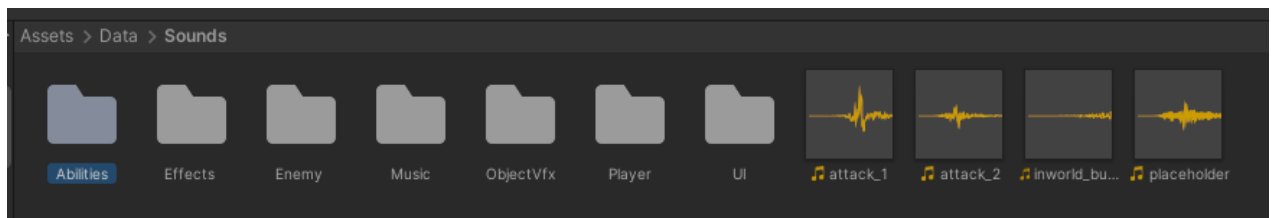


Рис. 22. Атлас звуков игры

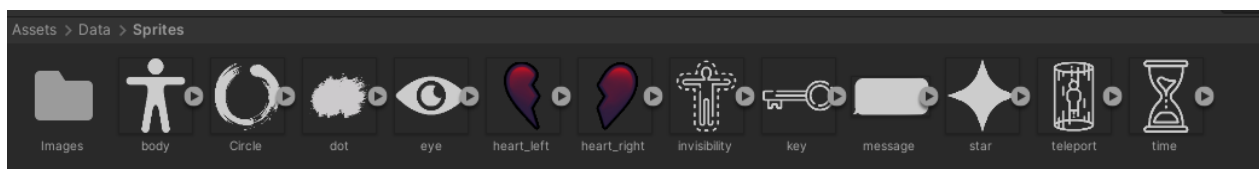


Рис.23. Атлас иконок игры

Модели экспортировались в формате FBX и настраивались в Unity:

1. Добавлены коллайдеры для взаимодействия с игровой физикой.
2. Настроены материалы и шейдеры для совместимости с динамическим освещением.
3. Модульные блоки лабиринта преобразованы в префабы с параметрами совместимости для WFC.

3.2. Разработка алгоритма генерации уровней

Каждый объект, участвующий в процедурной генерации, описывается набором параметров, обеспечивающих его корректное взаимодействие с окружением. Вес объекта задаёт вероятность его появления в уровне: чем выше значение, тем чаще элемент встречается в сгенерированном пространстве. Например, стандартные блоки лабиринта имеют максимальный вес, тогда как редкие элементы (углы, перекрестки) — минимальный.

Типы соединений формируют правила совместимости между объектами. Каждая грань элемента (верх, низ, право, лево) получает уникальное название-идентификатор (например, «проход», «воздух»). Соединение возможно только при полном совпадении идентификаторов у смежных граней. Это гарантирует органичное сочетание блоков: проходы стыкуются с проходами, а декоративные элементы — с совместимыми участками.

Шаблон модели включает 3D-меш, коллайдеры и прикрепленные скрипты, обеспечивая визуальную и физическую целостность объекта. Например, блок «перекрёсток» содержит четыре прохода с идентификаторами «проход» и модель с пересекающимися коридорами.

3.2.1. Настройка ячеек и ограничений

В данном фрагменте кода (листинг 6) прописывается описание каждого объекта, который будет участвовать в генерации. Задается вес каждого объекта и типы соединений, а также шаблон модели. Объекты могут соединиться только в том случае, если название соединения будет одинаковым.

```
using UnityEngine;
[CreateAssetMenu(fileName = "New Tile", menuName = "WFC/Tile")]
public class Tile : ScriptableObject
{
    public GameObject prefab;
    public float weight = 1f;
    public string upConnection = "pass";
    public string downConnection = "pass";
    public string leftConnection = "pass";
    public string rightConnection = "pass";
}
```

Листинг 6. Описание объектов

Шаблон закрепляет за собой алгоритм (листинг 7) и далее он определяет сколько создастся декоративных элементов на ячейке. Каждое место имеет шанс 50% что появится декорация. С 12,5% шансом декорация заменится на ловушку. Места, на которых может появиться объект определяются в среде разработки.

```

public class ObjectSpawner : MonoBehaviour
{
    [Header("Door")]
    [SerializeField] private bool spawnDoor = true;
    [SerializeField] private GameObject doorPrefab;
    [SerializeField] [Range(0f, 1f)] private float
chanceToSpawnDoor = 0.2f;
    [Header("Portal")]
    [SerializeField] private bool spawnPortal = true;
    [SerializeField] private GameObject portalPrefab;
    [SerializeField] [Range(0f, 1f)] private float
chanceToSpawnPortal = 0.25f;
}

```

Листинг 7. Создание объектов

Расставление интерактивных объектов. В данном фрагменте кода (листинг 8) происходит расставление ключей к уже созданным дверям. К каждой двери гарантированно должен идти ключ, но есть шанс получить дополнительный ключ.

```

public class WFCFinisher : MonoBehaviour
{
    public static WFCFinisher instance;
    private void Awake() { instance = this;}
    public void AddDoorSpawned() {
        doorsSpawned++;
        Instantiate(keyPrefab,
MeshGenerator.instance.GetRandomPosition(), Quaternion.identity);
        if (Random.Range(0f, 1f) < chanceToSpawnAdditionalKey)
        {
            Instantiate(keyPrefab,
MeshGenerator.instance.GetRandomPosition(), Quaternion.identity)
;
        }
    }
}

```

Листинг 8. Завершение генерации

3.2.2. Адаптация WFC для 3D-лабиринтов

Метод (листинг 9) заполняет каждую ячейку всеми возможными вариантами плиток. Когда сетка создалась, на поле в случайном месте создаются плитки начала и конца уровня.


```
void InitializeGrid()
{
    grid = new Tile[gridWidth, gridHeight];
    possibleTiles = new List<Tile>[gridWidth, gridHeight];
    for (int x = 0; x < gridWidth; x++)
    {
        for (int z = 0; z < gridHeight; z++)
        {
            possibleTiles[x, z] = new List<Tile>(tiles);
        }
    }
}
```

Листинг 9. Инициализация сетки

Алгоритм (листинг 10) начинает работу с выбора ячейки, обладающей наименьшим количеством возможных состояний. Это позволяет минимизировать неопределенность на ранних этапах и снизить риск противоречий в дальнейшем. Для выбранной ячейки осуществляется перебор доступных вариантов, после чего определяется окончательное состояние, соответствующее заданным правилам совместимости.

Далее код обновляет информацию в соседних ячейках, удаляя из их множества состояний те, которые конфликтуют с текущим выбором.

Процесс (листинг 11) повторяется циклически: на каждом шаге определяется новая ячейка с минимальным числом вариантов, её состояние фиксируется, а граничные элементы корректируются. Итерации продолжаются до тех пор, пока не будут обработаны все ячейки игрового поля. В итоге метод создает экземпляры объекта каждой ячейки на игровом поле.

```

void Generate()
{
    Vector2Int? lastCell = null;
    while (!IsGridFilled())
    {
        Vector2Int cell = FindLowestEntropyCell();
        if (cell.x == -1 || cell.y == -1 || lastCell == cell)
        {
            Debug.LogError("Generation failed :(");
            break;
        }
        lastCell = cell;
        CollapseCell(cell);
        Propagate(cell);
    }
    InstantiateGrid();
}

```

Листинг 10. Генерация

```

Vector2Int FindLowestEntropyCell(){
    Vector2Int cell = new Vector2Int(-1, -1);
    int minEntropy = int.MaxValue;
    for (int x = 0; x < gridWidth; x++) {
        for (int z = 0; z < gridHeight; z++) {
            if (grid[x, z] == null && possibleTiles[x, z].Count <
minEntropy) {
                minEntropy = possibleTiles[x, z].Count;
                cell = new Vector2Int(x, z);
            }
        }
    }
    return cell;
}

```

Листинг 11. Алгоритм выбора ячейки

Метод (листинг 12) выбирает случайную ячейку, подходящую по весам.

```
void CollapseCell(Vector2Int cell)
{
    List<Tile> possible = possibleTiles[cell.x, cell.y];
    if (possible.Count == 0)
    {
        Debug.LogError("No possible tiles to collapse at cell: " +
cell);
        return;
    }
    Tile selectedTile = GetWeightedRandomTile(possible);
    if (selectedTile == null)
    {
        Debug.LogError("Selected tile is null at cell: " + cell);
        return;
    }
    grid[cell.x, cell.y] = selectedTile;
    possibleTiles[cell.x, cell.y].Clear();
    possibleTiles[cell.x, cell.y].Add(selectedTile);
}
```

Листинг 12. Выбор ячейки

Алгоритм (листинг 13) рассчитывает суммарный вес всех доступных вариантов, генерирует случайное число в пределах этого диапазона и последовательно уменьшает веса, пока не подойдёт под значение веса текущей позиции из списка. Соответствующий вариант, на котором условие удовлетворено, фиксируется в качестве результата.

```
Tile GetWeightedRandomTile(List<Tile> tiles)
{
    float totalWeight = tiles.Sum(t => t.weight);
    float randomValue = UnityEngine.Random.Range(0, totalWeight);
    foreach (Tile tile in tiles)
    {
        if (randomValue < tile.weight)
        {
            return tile;
        }
        randomValue -= tile.weight;
    }
    return tiles[0];
}
```

Листинг 13. Выбор ячейки по случайному весу

Далее метод (листинг 14) сокращает множество значений соседних ячеек к выбранной.

```
void Propagate(Vector2Int cell)
{
    Queue<Vector2Int> queue = new Queue<Vector2Int>();
    queue.Enqueue(cell);
    while (queue.Count > 0)
    {
        Vector2Int currentCell = queue.Dequeue();
        Tile currentTile = grid[currentCell.x, currentCell.y];
        if (currentTile == null) continue;
        CheckNeighbor(currentCell, currentTile, new Vector2Int(1, 0),
queue);
        CheckNeighbor(currentCell, currentTile, new Vector2Int(-1, 0),
queue);
        CheckNeighbor(currentCell, currentTile, new Vector2Int(0, 1),
queue);
        CheckNeighbor(currentCell, currentTile, new Vector2Int(0, -1),
queue);
    }
}
```

Листинг 14. Сокращение вариантов

В конце алгоритм (листинг 15) создает экземпляры ячеек в случайном порядке для того, чтобы остальные скрипты работали корректно.

```
void InstantiateGrid(){
    List<Vector2Int> gridPositions = new List<Vector2Int>();
    for (int x = 0; x < gridWidth; x++){
        for (int z = 0; z < gridHeight; z++){
            gridPositions.Add(new Vector2Int(x, z));
        }
    }
    Shuffle(gridPositions);
    foreach (Vector2Int position in gridPositions)
    {
        Tile tile = grid[position.x, position.y];
        if (tile != null)
        {
            Instantiate(tile.prefab, new Vector3(position.x * 10, 0,
position.y * 10), tile.prefab.transform.rotation);
        }
    }
}
```

Листинг 15. Конец алгоритма

Алгоритм Wave Function Collapse был реализован для генерации 3D-лабиринтов на основе заданных правил совместимости. Каждый тайл получил уникальные идентификаторы соединений, что позволило обеспечить логическую стыковку элементов (проходы с проходами, стены со стенами). Алгоритм последовательно обрабатывал ячейки, начиная с наименьшей энтропией, фиксировал их состояние и распространял ограничения на соседние позиции, исключая конфликтующие варианты. Это обеспечило создание структур, соответствующих входным шаблонам, с учётом весов и геометрических особенностей. Для 3D-пространства были адаптированы проверки совместимости по двум осям, а также добавлена система случайного размещения декораций и ловушек, сохраняющая баланс между разнообразием и игровой логикой.

В результате работы алгоритма получается интересные варианты карт уровней (см. рис. 24) с уже расставленными ловушками, дверями, ключами и декоративными элементами. Сразу при запуске уровня пользователь будет видеть главного персонажа, стоящего на платформе (см. рис. 25).

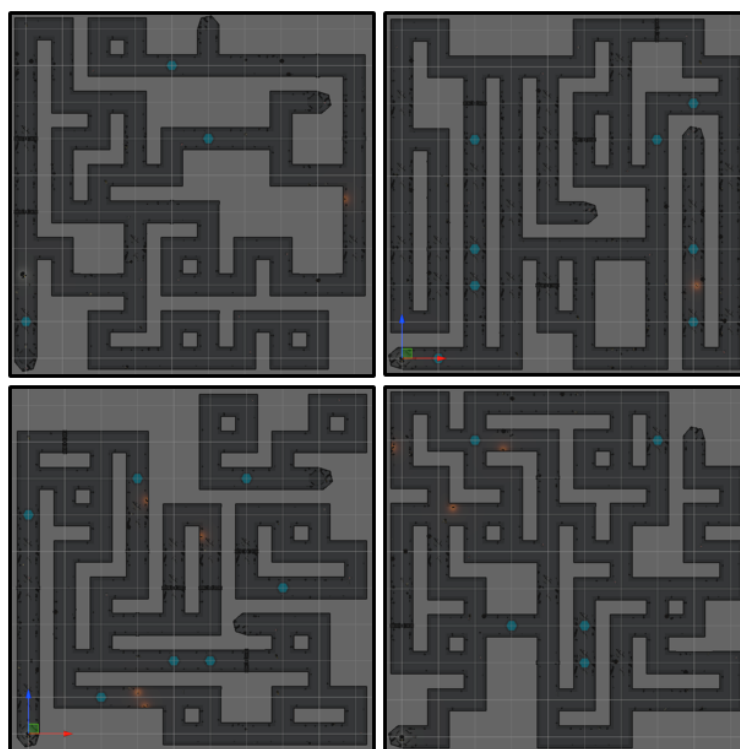


Рис. 24. Несколько вариантов карт уровня



Рис. 25. Вид уровня в игре

Некоторые настройки генератора (размер карты, количество дверей, максимальное количество телепортов, шанс появления ключа, количество лечащих зелий) меняются в инспекторе Unity (см. рис. 26).

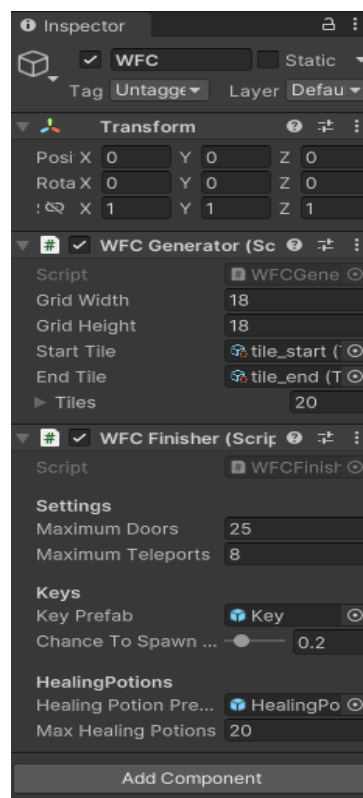


Рис. 26. Настройка WFC в инспекторе

3.3. Разработка пользовательского интерфейса

Проектирование пользовательского интерфейса (UI) ориентировалось на обеспечение функциональности и соответствие общей стилистике игры, где акцент сделан на баланс между практичностью и визуальной гармонией. Основной задачей стало создание системы, которая органично интегрируется в игровой процесс, не перегружая пользователя избыточной информацией.

Главное меню. В ходе разработки игры было доработано главное меню (см. рис. 27). Шрифты выбраны с учетом читаемости, а анимация плавного появления элементов снижает визуальную нагрузку.



Рис. 25. Главное меню

Игровое меню. Вертикальное меню (см. рис. 28) с кнопками «Продолжить», «Настройки», «В главное меню» и ползунками громкости обеспечивает логичную последовательность действий.

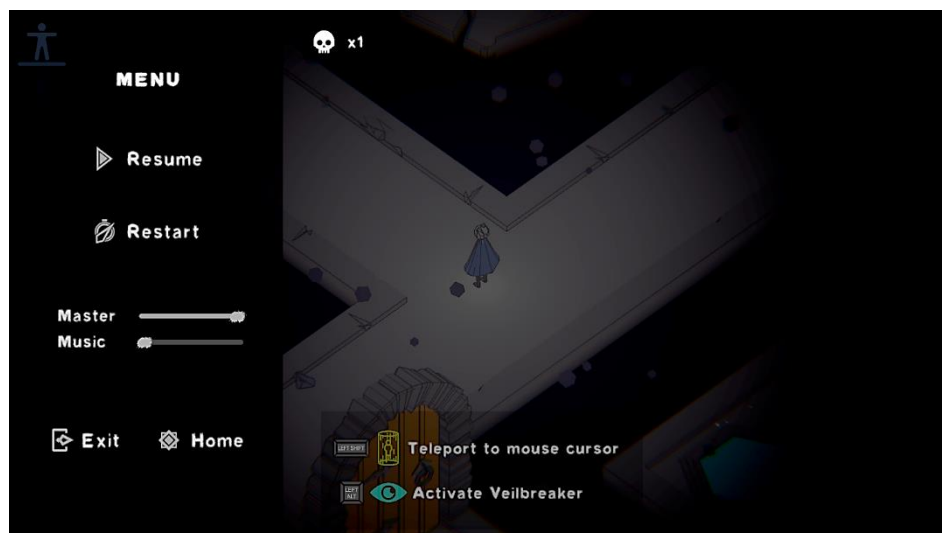


Рис. 26. Игровое меню

Экран поражения. Кнопки «Повторить», «Следующий уровень» и «Выход» размещены по центру, а фон слегка размыт, чтобы минимизировать отвлекающие факторы (см. рис. 29).

Обратная связь: Анимация плавного затемнения экрана и звуковые сигналы усиливают восприятие результатов игрового сеанса.

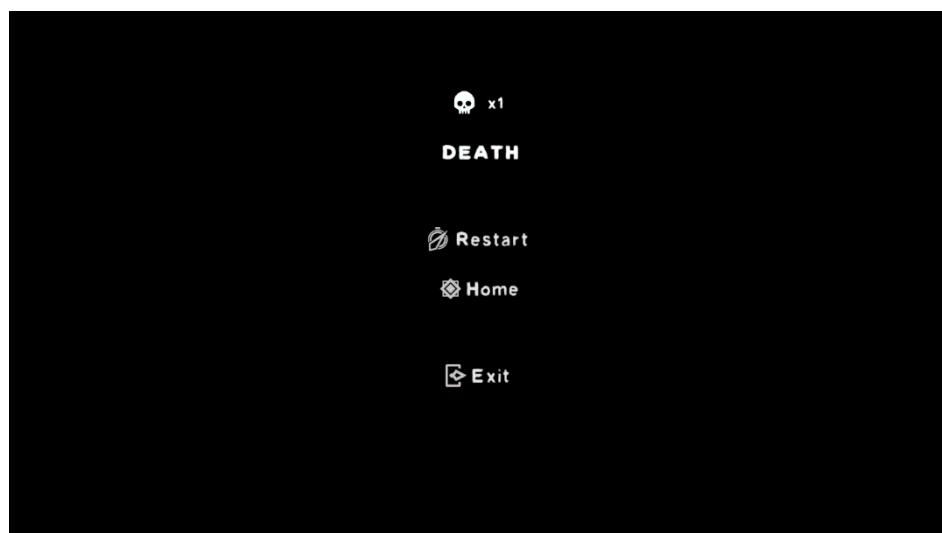


Рис. 27. Экран поражения

Интерфейс масштабируется под разные разрешения экрана, сохраняя пропорции и читаемость элементов.

3.4. Апробация

Апробация игры проводилась в рамках мероприятия «Майский Митап Индикатора», организованного для разработчиков и энтузиастов игровой индустрии. Целью тестирования стала оценка игрового опыта, выявление сильных сторон и проблемных зон, а также сбор обратной связи для дальнейшей доработки проекта.

Участникам мероприятия (15 человек) было предложено пройти несколько уровней, включающих основные механики: взаимодействие с порталами, преодоление ловушек и противостояние Всадникам. Среднее время сессии составило 15–20 минут. После завершения игрового процесса каждый участник заполнил анонимный опросник.

Вывод по результатам опроса:

Интерфейс и обучение. Многие пользователи (особенно не прошедшие обучение) испытывают трудности с пониманием интерфейса и механик игры. Описания способностей ("Pylon", "Veilbreaker", "Teleport") недостаточно ясны. Некоторые игроки не понимают, как их активировать и использовать.

Механики игры и уровни. Сложность уровней воспринимается неоднозначно: часть игроков считает их слишком запутанными, другие — слишком простыми. Ловушки и цели уровней часто непонятны (например, неочевидно, что является ловушкой, куда идти, как собрать ключи).

Визуальный стиль игры нашел удовлетворение у игроков, но есть запросы на большее разнообразие локаций и деталей.

На рисунках с 28 по 31 представлены количественные результаты оценки каждого критерия, а также распределение ответов на открытые вопросы, касающиеся предложений по улучшению.

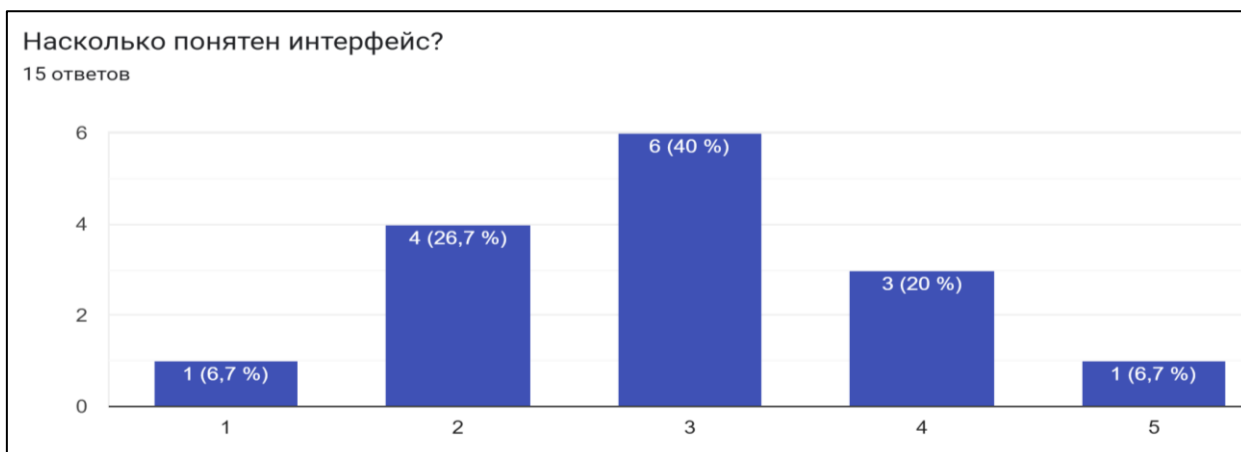


Рис. 28. Статистика по вопросу «Насколько понятен интерфейс?»

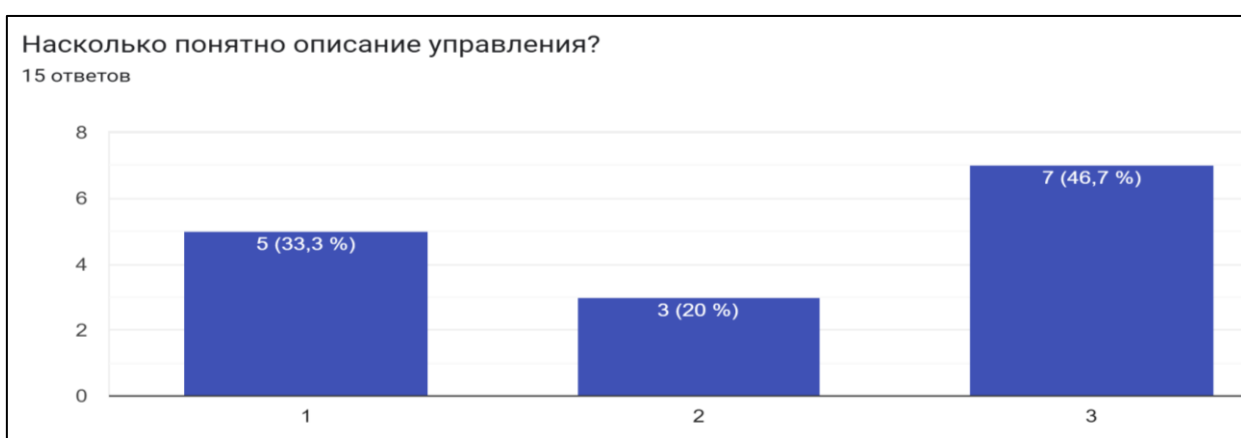


Рис. 29. Статистика по вопросу «Насколько понятно описание управления?»

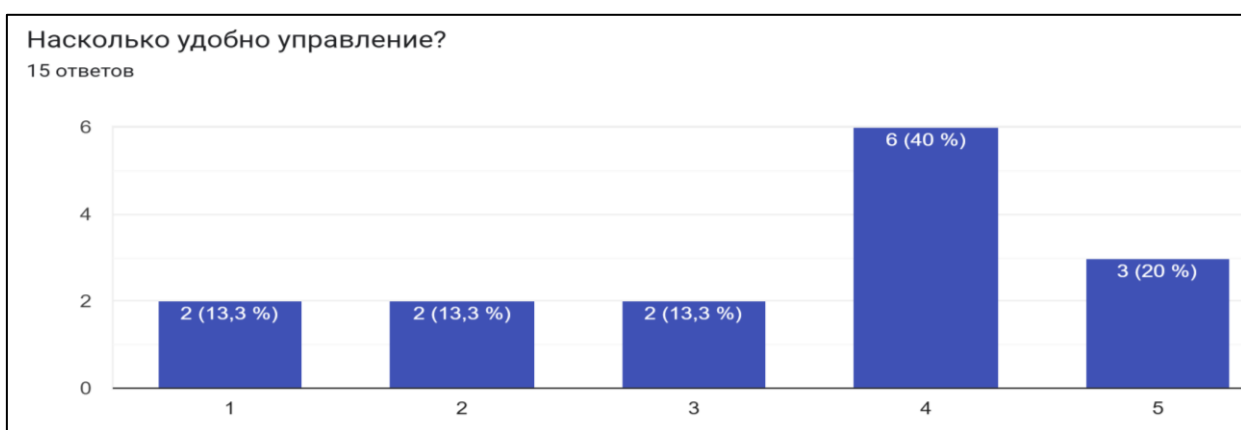


Рис. 30. Статистика по вопросу «Насколько удобно управление?»

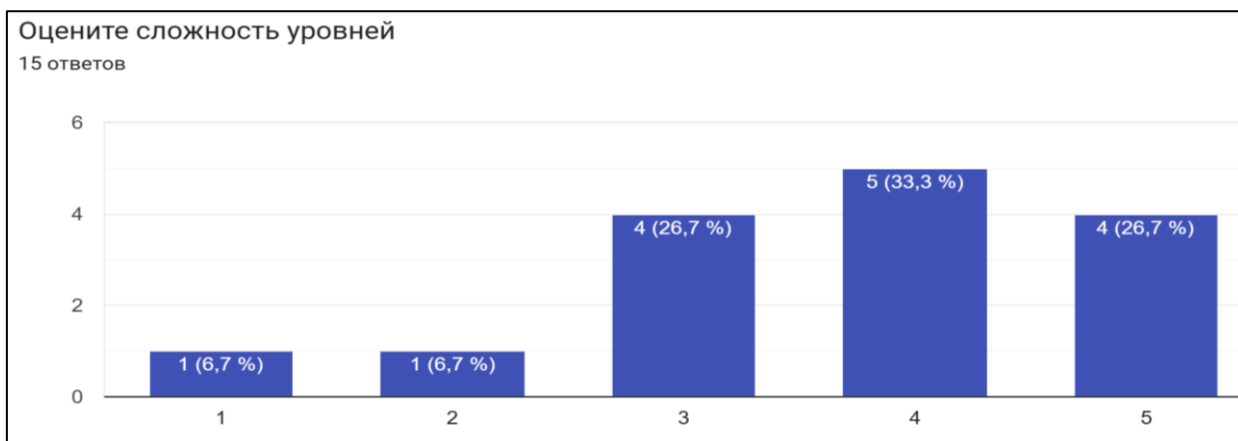


Рис. 31. Статистика по вопросу «Оцените сложность уровней»

Предложения по улучшению:

1. Ввести настройки яркости.
2. Добавить сюжетные элементы для повышения вовлеченности.
3. Добавить точки сохранения для исключения необходимости перепрохождения уровня с начала.
4. Сделать ловушки и цели более заметными (подсветка, анимации).
5. Переработать сложность уровней.
6. Добавить интерактивное обучение с визуальными подсказками.
7. Улучшить текстовые и визуальные описания навыков (например, добавить демонстрацию их эффектов).

Вывод: Игра имеет потенциал, но требует доработки в части обучения, балансировки уровней и улучшения удобства использования. Ключевые запросы игроков — понятность механик, удобство управления и визуальная обратная связь.

Вывод по третьей главе

В рамках третьей главы была успешно реализована ключевая механика игры, включая управление персонажем, взаимодействие с объектами, систему навыков и процедурную генерацию уровней на основе алгоритма Wave Function Collapse (WFC). Адаптация WFC для трёхмерных лабиринтов

позволила создавать структурно сложные и разнообразные уровни. Разработанные 3D-модели персонажей, врагов и игровых объектов, а также модульные блоки лабиринта, интегрированные в Unity, обеспечили соответствие визуального стиля заявленной атмосфере игры. Интерфейс, спроектированный с учётом минимализма, дополнил игровой опыт, а апробация выявила как сильные стороны проекта (вариативность уровней, атмосферность), так и зоны для улучшения (обучение, баланс сложности). Реализация подтвердила эффективность выбранного подхода к процедурной генерации и его потенциал для создания реиграбельного контента.

ЗАКЛЮЧЕНИЕ

Курсовая работа продемонстрировала возможности алгоритма Wave Function Collapse в контексте автоматической генерации уровней для игр жанра головоломка. В ходе исследования были решены поставленные задачи: проведён анализ существующих методов процедурной генерации, адаптирован WFC для трёхмерных пространств, разработан прототип игры с динамически создаваемыми лабиринтами и проведена оценка его играбельности. Практическая значимость работы заключается в сокращении временных затрат на ручное проектирование уровней и повышении уникальности игрового опыта за счёт процедурного подхода.

Результаты апробации подтвердили, что алгоритм WFC способен генерировать уровни, сочетающие структурную сложность и органичность, однако выявили необходимость доработки обучения игрока и балансировки механик. Перспективы дальнейших исследований включают оптимизацию алгоритма для больших карт, интеграцию машинного обучения для персонализации уровней и расширение функционала генератора под задачи других жанров. Таким образом, работа вносит вклад в развитие методов автоматизации игрового дизайна, открывая новые возможности для создания динамического и адаптивного контента.

СПИСОК ЛИТЕРАТУРЫ

1. WaveFunctionCollapse // GitHub URL:
<https://github.com/mxgmn/WaveFunctionCollapse> (дата обращения: 27.10.2024).
2. Alexei A. Efros and Thomas K. Leung Texture Synthesis by Non-parametric Sampling. *Computer Science Division*. 1999 г. - 6 с.
3. Проледурное создание бесшовного города // XYZ Media URL:
<https://media-xyz.com/ru/articles/1195-gaid-protsedurnoe-sozdanie-besshovnogo-goroda> (дата обращения: 27.10.2024).
4. Алгоритмы поиска пути. // PMG URL:
<https://pmg.org.ru/ai/stout.htm> (дата обращения: 27.10.2024).
5. Генерация лабиринта и его прохождение // progressor-blog URL:
<https://progressor-blog.ru/qt/generatsiya-labirinta-i-ego-prohozhdenie/> (дата обращения: 27.10.2024).
6. Maze Classification // astrolog URL:
<https://www.astrolog.org/labyrnth/algrithm.htm#perfect> (дата обращения: 27.10.2024).
7. Documentation // DOTween URL:
<https://dotween.demigiant.com/documentation.php> (дата обращения: 19.05.2025).

ПРИЛОЖЕНИЕ 1

Скрипт механики управления пилонами.

```
using System.Collections.Generic;
using DG.Tweening;
using UnityEngine;
using UnityEngine.InputSystem;

public class PylonsAbility : MonoBehaviour
{
    [SerializeField] private List<GameObject>
availablePylonPrefabs;
    [SerializeField] private List<GameObject> particles;
    [SerializeField] private List<GameObject> pylons;
    [SerializeField] private int maxPlacablePylons = 5;
    [SerializeField] private AudioSource audioSource;
    [SerializeField] private AudioClip directionEffectSound;
    [SerializeField] private float cooldown = 1f;
    private float cooldownTimer = 0f;

    private void Start()
    {
        if (particles.Count != availablePylonPrefabs.Count)
        {
            Debug.LogWarning("Not enough particles to enable
pylons in PylonsAbility.cs");
            enabled = false;
            return;
        }

        if (availablePylonPrefabs.Count < maxPlacablePylons)
        {
            Debug.LogWarning("Not enough pylons in
PylonsAbility.cs");
            maxPlacablePylons = availablePylonPrefabs.Count;
        }

        for (int i = 0; i < maxPlacablePylons; i++)
        {
            pylons.Add(null);
        }

        PylonsAmountChnaged();

        if (audioSource == null) audioSource =
GetComponent<AudioSource>();
    }

    private void Update()
    {
        if (cooldownTimer > 0f)
        {
            cooldownTimer -= Time.deltaTime;
        }
    }
}
```

```

        if (Keyboard.current.zKey.wasReleasedThisFrame)
PlacePylon();
        if (Keyboard.current.xKey.wasReleasedThisFrame)
ResetPylons();
        if (Keyboard.current.cKey.wasReleasedThisFrame)
ShowDirections();
    }

    public void PlacePylon()
    {
        if (GetPlacedPylonsCount() < maxPlacablePylons)
        {
            int availableIndex = GetFirstAailableIndex();
            if (availableIndex == -1) return;

            GameObject pylon =
Instantiate(availablePylonPrefabs[availableIndex],
transform.position, Quaternion.identity);
            if (!GameEvents.current.isTopSurface)
pylon.transform.rotation = Quaternion.Euler(180, 0, 0);
            pylon.GetComponent<Pylon>().pylonIndex =
availableIndex;
            pylons[availableIndex] = pylon;

            PylonsAmountChnaged();
        }
    }

    private void PylonsAmountChnaged()
    {
        GameEvents.current.PylonsAmountChanged(maxPlacablePylons
- GetPlacedPylonsCount());
    }

    private int GetPlacedPylonsCount()
    {
        int count = 0;
        foreach (GameObject pylon in pylons)
        {
            if (pylon != null) count++;
        }
        return count;
    }

    private int GetFirstAailableIndex()
    {
        for (int i = 0; i < availablePylonPrefabs.Count; i++)
        {
            if (pylons[i] == null)
            {
                return i;
            }
        }
        return -1;
    }

    public void CollectPylon(int index)

```



```

    {
        if (index < 0 || index >= pylons.Count) return;
        pylons[index] = null;
        PylonsAmountChnaged();
    }

    public void ResetPylons()
    {
        bool soundMade = false;
        for (int i = 0; i < pylons.Count; i++)
        {
            if (pylons[i] == null) continue;
            pylons[i].GetComponent<Pylon>().InteractWithSound(gameObject, !soundMade);
            soundMade = true;
        }
    }

    public void ShowDirections()
    {
        if (GetPlacedPylonsCount() < 1) return;

        if (cooldownTimer > 0f) return;
        cooldownTimer = cooldown;

        if (directionEffectSound != null)
            audioSource.PlayOneShot(directionEffectSound, 0.1f);

        for (int i = 0; i < pylons.Count; i++)
        {
            if (pylons[i] == null) continue;

            GameObject particle = Instantiate(particles[i],
            transform.position, Quaternion.identity);

            float distance =
            Vector3.Distance(particle.transform.position,
            pylons[i].transform.position);
            particle.transform.DOMove(pylons[i].transform.position,
            distance * 0.25f).OnComplete(() =>
            {
                particle.GetComponent<ParticleSystem>().Stop();
                Destroy(particle,
                particle.GetComponent<ParticleSystem>().main.startLifetime.constant);
            });
        }
    }
}

```

ПРИЛОЖЕНИЕ 2

Скрипт системы навыков персонажа.

```
using System.Collections.Generic;
using DG.Tweening;
using Unity.VisualScripting;
using UnityEditor;
using UnityEngine;
using UnityEngine.AI;
using UnityEngine.InputSystem;

public class Abilities : MonoBehaviour
{
    [Header("Invisibility")]
    [SerializeField] private bool isInvisibilityEnabled = true;
    [SerializeField] private AudioClip invisibilitySound;
    [SerializeField] private float invisibilityActivationTime =
0.75f;
    [SerializeField] private float invisibilityDuration = 2f;
    [SerializeField] private float invisibilityDelay = 30f;
    private float invisibilityLastTimeUsed = float.MinValue;
    private bool isInvisible = false;
    public bool IsInvisible => isInvisible;
    public bool IsInvisibilityUnlocked => isInvisibilityEnabled;

    [Header("Wide Vision")]
    [SerializeField] private bool isWideVisionEnabled = true;
    [SerializeField] private AudioClip wideVisionSound;
    [SerializeField] private AudioClip wideVisionSoundEnd;
    [SerializeField] private float wideVisionDuration = 3.5f;
    [SerializeField] private float wideVisionDelay = 20f;
    private float wideVisionLastTimeUsed = float.MinValue;
    public bool IsWideVisionUnlocked => isWideVisionEnabled;

    [Header("Teleport")]
    [SerializeField] private bool isTeleportEnabled = true;
    [SerializeField] private AudioClip teleportSound;
    [SerializeField] private int teleportationCount = 5;
    [SerializeField] private float teleportationRadius = 10f;
    [SerializeField] private float teleportationDelay = 3f;
    [SerializeField] private bool teleportationShouldRefill =
true;
    [SerializeField] private float teleportationRefillDelay =
45f;
    private float teleportationLastTimeUsed = float.MinValue;
    public bool IsTeleportUnlocked => isTeleportEnabled;

    [Header("Other")]
    [SerializeField] private SkinnedMeshRenderer playerModel;
    [SerializeField] private AudioClip cdSound;
    private bool isUsingAbility = false;
    public bool IsUsingAbility => isUsingAbility;
    private List<Material> materials;
    private AudioSource audioSource;
    private Camera cam;
```

```

void Start()
{
    materials = new List<Material>();
    materials.AddRange(playerModel.materials);
    audioSource = GetComponent<AudioSource>();
    cam = Camera.main;
}

void Update()
{
    if (isUsingAbility) return;

    if (Keyboard.current.ctrlKey.wasReleasedThisFrame &&
isInvisibilityEnabled) HandleInvisibility();
    if (Keyboard.current.altKey.wasReleasedThisFrame &&
isWideVisionEnabled) HandleWideWision();
    if (Keyboard.current.shiftKey.wasReleasedThisFrame &&
isTeleportEnabled) HandleTeleport();
}

/// <summary>
/// Handles the invisibility ability
/// </summary>
public void StartInvisibilityExternal()
{
    isUsingAbility = true;
    StartInvisibility(false);
}

private void HandleInvisibility()
{
    if (Time.time < invisibilityLastTimeUsed +
invisibilityDelay) return;
    isUsingAbility = true;
    StartInvisibility();
}

private void StartInvisibility(bool shouldGoOnCd = true)
{
    isInvisible = true;
    audioSource.PlayOneShot(invisibilitySound);
    foreach (var material in materials)
    {
        DOTween.To(() => material.GetFloat("_Dissolve"), x =>
material.SetFloat("_Dissolve", x), 1,
invisibilityActivationTime).SetEase(Ease.InOutQuart);
    }
    DOVirtual.DelayedCall(invisibilityActivationTime +
invisibilityDuration, () => EndInvisibility(shouldGoOnCd));
}

private void EndInvisibility(bool shouldGoOnCd = true)
{
    foreach (var material in materials)
    {

```

```

        DOTween.To(() => material.GetFloat("_Dissolve"), x =>
material.SetFloat("_Dissolve", x), 0,
invisibilityActivationTime).SetEase(Ease.InOutQuart);
    }
    DOVirtual.DelayedCall(invisibilityActivationTime, () =>
OnInvisibilityEnded(shouldGoOnCd));
    }

    private void OnInvisibilityEnded(bool shouldGoOnCd = true)
    {
        isInvisible = false;
        isUsingAbility = false;
        if (!shouldGoOnCd) return;
        GameEvents.current.InvisibilityCD(invisibilityDelay);
        DOVirtual.DelayedCall(invisibilityDelay, () =>
PlayCDSound());
        invisibilityLastTimeUsed = Time.time;
    }

    /// <summary>
    /// Handles the wide vision ability
    /// </summary>
    private void HandleWideWision()
    {
        if (Time.time < wideVisionLastTimeUsed + wideVisionDelay)
return;

        isUsingAbility = true;
        audioSource.PlayOneShot(wideVisionSound);
        GameEvents.current.WideVisionUsed(wideVisionDuration);
        GameEvents.current.WideVisionCD(wideVisionDelay);
        DOVirtual.DelayedCall(wideVisionDelay, () =>
PlayCDSound());
        wideVisionLastTimeUsed = Time.time;
        DOVirtual.DelayedCall(wideVisionDuration * 0.65f, () =>
audioSource.PlayOneShot(wideVisionSoundEnd));

        isUsingAbility = false;
    }

    /// <summary>
    /// Handles the teleport ability
    /// </summary>
    private void HandleTeleport()
    {
        if (isUsingAbility) return;
        if (Time.time < teleportationLastTimeUsed +
teleportationDelay) return;
        if (teleportationCount > 0)
        {
            isUsingAbility = true;
            Teleport();
        }
    }

    private void Teleport()
    {

```

```

        Ray ray = cam.ScreenPointToRay(Input.mousePosition);
        if (Physics.Raycast(ray, out RaycastHit hitPoint))
        {
            Vector3 tpPosition = hitPoint.point;
            if (NavMesh.SamplePosition(tpPosition, out NavMeshHit
hit, 1f, NavMesh.AllAreas))
            {
                if (hit.position.y > 0 &&
GameEvents.current.isTopSurface || hit.position.y < 0 &&
!GameEvents.current.isTopSurface)
                {
                    tpPosition = hit.position;
                    tpPosition.y =
gameObject.transform.position.y;
                    if (Vector3.Distance(transform.position,
tpPosition) < teleportationRadius)
                    {
                        GameEvents.current.PlayerUsedTeleport(tpP
osition);

                        audioSource.PlayOneShot(teleportSound);
                        teleportationCount--;
                        GameEvents.current.TeleportAmountChanged(
teleportationCount);

                        float delay = teleportationCount == 0 &&
teleportationShouldRefill ? teleportationRefillDelay :
teleportationDelay;

                        GameEvents.current.TeleportCD(delay);
                        DOVirtual.DelayedCall(delay, () =>
                        {
                            if (teleportationCount == 0 &&
teleportationShouldRefill) RefillTeleportationCount();
                            PlayCDSound();
                        });
                        teleportationLastTimeUsed = Time.time;
                    }
                }
            }
        }
        isUsingAbility = false;
    }
    private void RefillTeleportationCount()
    {
        teleportationCount = 5;
        GameEvents.current.TeleportAmountChanged(teleportationCou
nt);

        GameEvents.current.TeleportsRefilled();
    }
    /// <summary>
    /// Other
    /// </summary>
    private void PlayCDSound()
    {
        audioSource.PlayOneShot(cdSound, 3f);
    }
    #if UNITY_EDITOR
    void OnDrawGizmosSelected()
    {

```

```
        Handles.color = Color.magenta;
        Handles.DrawWireDisc(transform.position - new Vector3(0,
GetComponent<Collider>().bounds.extents.y, 0), Vector3.up,
teleportationRadius);
    }
#endif
}
```

ПРИЛОЖЕНИЕ 3

Скрипт реализации механики атаки нимбом.

```
using DG.Tweening;
using UnityEngine;

public class PlayerHalo : MonoBehaviour
{
    private Transform playerTransform;
    private Camera cam;
    private Material material;

    [Header("Settings")]
    [SerializeField] private Vector3 offset = new Vector3(0,
1.25f, 0);
    [SerializeField] private float smoothSpeed = 0.1f;
    [SerializeField] private float angle = 20f;

    [Header("Functionality")]
    [SerializeField] private float attackRange = 5f;
    [SerializeField] private float attackMinimumRange = 1.5f;
    [SerializeField] private float attackStart = 0.35f;
    [SerializeField] private float attackSpeed = 0.5f;
    [SerializeField] private float attackCD = 3f;
    private float attackCDTimer = 0f;
    private bool isAttacking = false;
    private BoxCollider col;

    public bool IsOnCooldown => Time.time < attackCDTimer +
attackCD;

    void Start()
    {
        playerTransform = GameObject.Find("Player").transform;
        col = GetComponent<BoxCollider>();
        cam = Camera.main;
        material = GetComponent<MeshRenderer>().material;
        attackCDTimer = -attackCD;

        GameEvents.current.onPlayerDied += () => isAttacking =
true;
    }

    void Update()
    {
        if (isAttacking || Time.time < attackCDTimer + attackCD)
return;
        if (Input.GetMouseButtonDown(0))
        {
            Ray ray = cam.ScreenPointToRay(Input.mousePosition);
            if (Physics.Raycast(ray, out RaycastHit hitPoint,
1000f))
            {
                float dist =
Vector3.Distance(playerTransform.position, hitPoint.point);
```

```

        if (dist < attackRange && dist >
attackMinimumRange)
        {
            Attack(hitPoint.point);
        }
    }
}

void Attack(Vector3 targetPosition)
{
    isAttacking = true;
    col.enabled = true;
    Sequence attackSequence = DOTween.Sequence();
    GetComponent().Play();
    attackSequence.Append(transform.DORotate(new Vector3(90,
0, 0), attackStart))
        .Append(transform.DOMove(targetPosition,
attackSpeed))
        .Join(transform.DORotate(new Vector3(0,
Random.Range(340, 540), 0), attackSpeed).SetRelative())
        .Append(DOTween.To(() =>
material.GetFloat("_Dissolve"), x =>
material.SetFloat("_Dissolve", x), 1,
0.5f).SetEase(Ease.InOutQuart))
        .OnComplete(() => {
            col.enabled = false;
            foreach (Transform child in transform)
            {
                Light light =
child.GetComponent<Light>();
                if (light != null) DOTween.To(() =>
light.intensity, x => light.intensity = x, 0,
0.5f).SetEase(Ease.InOutQuart);
            }
            isAttacking = false;
            attackCDTimer = Time.time;
            DOVirtual.DelayedCall(attackCD - 0.5f,
() => {
                DOTween.To(() =>
material.GetFloat("_Dissolve"), x =>
material.SetFloat("_Dissolve", x), 0,
0.5f).SetEase(Ease.InOutQuart);
                foreach (Transform child in
transform)
                {
                    Light light =
child.GetComponent<Light>();
                    if (light != null)
DOTween.To(() => light.intensity, x => light.intensity = x, 0.5f,
0.5f).SetEase(Ease.InOutQuart);
                }
            }));
        });
}

```



```

void LateUpdate()
{
    if (isAttacking) return;

    Vector3 targetPosition = playerTransform.position -
playerTransform.forward * 0.8f;
    targetPosition += GameEvents.current.isTopSurface ?
offset : -offset;
    transform.position = Vector3.Lerp(transform.position,
targetPosition, smoothSpeed);

    Quaternion targetRotation = playerTransform.rotation *
Quaternion.Euler(angle, 0, 0);
    transform.rotation = Quaternion.Lerp(transform.rotation,
targetRotation, smoothSpeed);
}
}

```